



# SafeSpace

**AI-Powered Mental Application Using Natural Language  
Processing And Diagnostic Questionnaires**

**Prepared by**

**Ali Ebrahim Sakr   Tasneem Mohamed   Mahmoud AboElNaga**

**Ahmad Behiry   Rowayda El-Bayar   Mohamed Sabry**

**Ali Saad Ali   Jana Khaled Awad   Youssef Ehab Gomaa**

**Supervisors**

**Assoc. Prof. Doaa Adel**

**Eng. Rana Amr Mohy**

**Academic Year: 2025 – 2026**

**June 2026**

## Abstract

Mental health disorders, particularly anxiety, depression, and stress, represent a significant global challenge that is frequently exacerbated by barriers such as social stigma, prohibitive therapeutic costs, and systemic shortages of professional clinicians. While traditional diagnostic tools like the Depression, Anxiety, and Stress Scale (DASS-42) provide clinically validated baselines, they are vulnerable to response bias or conscious “masking” by users who manipulate their selections to appear more resilient. Conversely, qualitative self-expression, such as daily journaling, offers rich contextual insights into a user’s emotional state but lacks objective clinical scoring.

This project introduces SafeSpace, a cross-platform mobile ecosystem designed to bridge the gap between objective clinical diagnostics and subjective emotional expression. SafeSpace leverages a multi-modal assessment engine that fuses quantitative inputs from a DASS-42 digital questionnaire with qualitative natural language processing (NLP) of bilingual (Arabic/English) journal entries. The underlying architecture employs a Multi-Layer Perceptron (MLP) for rapid clinical survey prediction alongside a fine-tuned cross-lingual transformer model (XLM-RoBERTa) to analyze the emotional tone and semantic context of colloquial Arabic dialects and English text.

A score-fusion algorithm integrates these modalities, weighting the clinical survey at 60% and the semantic journal analysis at 40% to compute a contextualized emotional profile. Furthermore, the system incorporates an automated lexicon-based parser to identify root stressors (e.g., academic, workplace, or relationship stress) and a real-time crisis detection safety net to bypass scoring and immediately trigger support interventions when self-harm markers are identified. The proposed solution offers a scalable, private, and culturally adaptive framework for early mental health screening and personalized coping support.

## Acknowledgements

We would like to express our sincere gratitude to our project supervisors for the continuous guidance, constructive feedback, and academic mentorship provided throughout every phase of this project. Their expertise in machine learning and software engineering was instrumental in shaping both the technical architecture and the research methodology of SafeSpace.

We extend our appreciation to the faculty members and teaching assistants of the department for the foundational knowledge they provided across courses in artificial intelligence, software engineering, database systems, and mobile application development. These courses formed the technical backbone upon which this project was built.

Finally, we are deeply grateful to our families and friends for their patience, encouragement, and unwavering support throughout the demanding months of development, testing, and documentation. This project would not have been possible without their presence.

## Table of Contents

|                                                                       |      |
|-----------------------------------------------------------------------|------|
| Abstract .....                                                        | ii   |
| Acknowledgements .....                                                | iii  |
| Table of Contents .....                                               | iv   |
| List of Figures .....                                                 | vii  |
| List of Abbreviations .....                                           | viii |
| Chapter 1: Introduction .....                                         | 1    |
| 1.1 Overview .....                                                    | 1    |
| 1.2 Problem Definition .....                                          | 1    |
| 1.3 Project Objectives .....                                          | 2    |
| 1.4 Project Scope .....                                               | 2    |
| 1.5 Timeline .....                                                    | 2    |
| Chapter 2: Literature Review .....                                    | 4    |
| 2.1 Introduction .....                                                | 4    |
| 2.2 Background .....                                                  | 4    |
| 2.2.1 Psychometric Screening and the DASS Tradition .....             | 4    |
| 2.2.2 Transformer Architectures for Text Classification .....         | 4    |
| 2.2.3 Challenges of Arabic Dialect Processing .....                   | 5    |
| 2.3 Relevant Works .....                                              | 5    |
| 2.4 Relationship Between Existing Work and Our Work .....             | 5    |
| 2.5 Summary .....                                                     | 6    |
| Chapter 3: Artificial Intelligence Module .....                       | 7    |
| 3.1 Introduction .....                                                | 7    |
| 3.1.1 Artificial Intelligence for Early Mental Health Screening ..... | 7    |
| 3.1.2 Ethical Considerations in AI-Based Mental Health Systems .....  | 7    |
| 3.2 Clinical Grounding: DASS-42 Implementation Specification .....    | 8    |
| 3.2.1 Severity Cutoffs as Implemented .....                           | 8    |
| 3.2.2 Item-to-Subscale Mapping .....                                  | 8    |
| 3.3 Datasets .....                                                    | 9    |
| 3.3.1 Survey Dataset — Open DASS-42 Responses .....                   | 9    |
| 3.3.2 NLP Dataset — MentalHealth-Darija Corpus .....                  | 10   |
| 3.3.3 Exploratory Data Analysis (EDA) .....                           | 10   |
| 3.4 Survey Model — Deep Neural Network on DASS-42 Responses .....     | 11   |
| 3.4.1 Problem Formulation .....                                       | 11   |
| 3.4.2 Data Preprocessing .....                                        | 11   |
| 3.4.3 Model Architecture .....                                        | 12   |
| 3.4.4 Training Configuration .....                                    | 13   |
| 3.4.5 Evaluation Results .....                                        | 14   |
| 3.5 NLP Model — Fine-Tuned XLM-RoBERTa for Text Classification .....  | 14   |
| 3.5.1 Model Selection Rationale .....                                 | 14   |
| 3.5.2 Input Representation .....                                      | 15   |
| 3.5.3 Preprocessing and Dataset Preparation .....                     | 15   |

---

|                                                     |    |
|-----------------------------------------------------|----|
| 3.5.4 Class Imbalance Handling                      | 15 |
| 3.5.5 Fine-Tuning Configuration                     | 16 |
| 3.5.6 Evaluation Results                            | 16 |
| 3.5.7 Keyword-Override Mechanism                    | 17 |
| 3.6 Model Fusion Strategy                           | 17 |
| 3.6.1 Motivation for Ensemble Fusion                | 17 |
| 3.6.2 Weighted Linear Fusion                        | 18 |
| 3.6.3 Score Transparency                            | 18 |
| 3.7 Recommendation System                           | 18 |
| 3.7.1 System Architecture                           | 18 |
| 3.7.2 Severity Classification                       | 18 |
| 3.7.3 Contextual Cause Extraction                   | 19 |
| 3.7.4 Suicidal Ideation Detection                   | 19 |
| 3.7.5 Recommendation Content                        | 19 |
| 3.8 System Integration and Deployment Architecture  | 19 |
| 3.9 Chapter Summary                                 | 20 |
| Chapter 4: System Analysis.....                     | 22 |
| 4.1 Development Methodology                         | 22 |
| 4.2 Requirements Analysis                           | 22 |
| 4.3 Functional Requirements                         | 22 |
| 4.4 Non-Functional Requirements                     | 23 |
| 4.5 System Workflow                                 | 23 |
| Chapter 5: System Design.....                       | 24 |
| 5.1 Introduction                                    | 24 |
| 5.2 Application Navigation Structure                | 24 |
| 5.3 Screen-by-Screen Design                         | 24 |
| 5.3.1 Splash and Onboarding Screens                 | 24 |
| 5.3.2 Registration and Authentication Screens       | 24 |
| 5.3.3 Home Dashboard                                | 25 |
| 5.3.4 Daily Check-In Screen                         | 25 |
| 5.3.5 DASS-42 Assessment Screen                     | 25 |
| 5.3.6 DASS Results Screen                           | 25 |
| 5.3.7 NLP Text Analysis Screen                      | 25 |
| 5.3.8 Journal Editor Screen                         | 26 |
| 5.3.9 Wellness Tools and Mini-Games                 | 26 |
| 5.3.10 Morning Ritual and Nightly Unwind Screens    | 26 |
| 5.3.11 Mood Patterns and Trend Visualization Screen | 26 |
| 5.3.12 Settings Screen                              | 26 |
| 5.4 Database Schema Design                          | 27 |
| 5.5 System Architecture Overview                    | 27 |
| 5.6 Accessibility and Responsive Design             | 29 |
| 5.7 Security and Privacy Considerations             | 29 |
| 5.8 Chapter Summary                                 | 29 |
| Chapter 6: Technical Implementation .....           | 30 |
| 6.1 Mobile Frontend Module                          | 30 |

---

|                                             |    |
|---------------------------------------------|----|
| 6.2 Backend API Module                      | 30 |
| 6.3 AI Model Implementations                | 31 |
| 6.3.1 DASS-42 Multi-Layer Perceptron        | 31 |
| 6.3.2 XLM-RoBERTa Text Classifier           | 32 |
| 6.3.3 Score Fusion and Crisis Override      | 33 |
| 6.4 Database and Authentication Module      | 34 |
| 6.5 Bilingual Recommendation Engine         | 34 |
| 6.6 Cross-Platform Flutter Web Deployment   | 35 |
| Chapter 7: Conclusion and Future Work ..... | 37 |
| 7.1 Conclusion                              | 37 |
| 7.2 Future Work                             | 37 |
| References .....                            | 39 |

## List of Figures

| Figure      | Description                                                   |
|-------------|---------------------------------------------------------------|
| Figure 4.1  | System workflow sequence diagram (DASS-42 + journal analysis) |
| Figure 5.1  | Splash and Onboarding Screen                                  |
| Figure 5.2  | Sign-up screen                                                |
| Figure 5.3  | Sign-in screen                                                |
| Figure 5.4  | Home screen                                                   |
| Figure 5.5  | Daily Check-In Screen                                         |
| Figure 5.6  | assessment boarding Screen                                    |
| Figure 5.7  | survey q16 Screen                                             |
| Figure 5.8  | assessment Results tips                                       |
| Figure 5.9  | assessment Results overview                                   |
| Figure 5.10 | NLP Analysis Screen                                           |
| Figure 5.11 | NLP with Text Analysis Screen(2)                              |
| Figure 5.12 | Journal Editor Screen                                         |
| Figure 5.13 | Wellness Menu Screen                                          |
| Figure 5.14 | meditation Screen                                             |
| Figure 5.15 | Morning Ritual Screen                                         |
| Figure 5.16 | Nightly Unwind Screen                                         |
| Figure 5.17 | Mood Patterns Screen(1)                                       |
| Figure 5.18 | Mood Patterns Screen(2)                                       |

|             |                                                                |
|-------------|----------------------------------------------------------------|
| Figure 5.19 | Profile Screen                                                 |
| Figure 5.20 | Settings Screen                                                |
| Figure 5.21 | SafeSpace high-level system architecture (four-layer overview) |



## List of Tables

| Table            | Description                                                                 |
|------------------|-----------------------------------------------------------------------------|
| <b>Table 1.1</b> | Project execution timeline across the academic year                         |
| <b>Table 3.1</b> | DASS-42 Official Severity Cutoff Ranges                                     |
| <b>Table 3.2</b> | Descriptive statistics of DASS-42 subscale raw scores                       |
| <b>Table 3.3</b> | Class distribution of the filtered MentalHealth-Darija NLP training dataset |
| <b>Table 3.4</b> | Survey DNN architecture layer configuration                                 |
| <b>Table 3.5</b> | Survey DNN evaluation metrics on the held-out test set                      |
| <b>Table 3.6</b> | XLM-RoBERTa fine-tuning hyperparameter configuration                        |
| <b>Table 3.7</b> | XLM-RoBERTa per-class classification report on the validation set           |
| <b>Table 5.1</b> | SafeSpace PostgreSQL schema overview                                        |
| <b>Table 6.1</b> | SafeSpace backend REST API endpoints                                        |
| <b>Table 6.2</b> | Bilingual cause-extraction keyword lexicon                                  |
| <b>Table 6.3</b> | Recommendation database coverage by condition and cause                     |

## List of Abbreviations

|                |                                                         |
|----------------|---------------------------------------------------------|
| <b>AI</b>      | Artificial Intelligence                                 |
| <b>API</b>     | Application Programming Interface                       |
| <b>BERT</b>    | Bidirectional Encoder Representations from Transformers |
| <b>CBT</b>     | Cognitive Behavioral Therapy                            |
| <b>CDN</b>     | Content Delivery Network                                |
| <b>DASS-42</b> | Depression, Anxiety, and Stress Scale (42-item version) |
| <b>DNN</b>     | Deep Neural Network                                     |
| <b>MAE</b>     | Mean Absolute Error                                     |
| <b>MLP</b>     | Multi-Layer Perceptron                                  |
| <b>MSA</b>     | Modern Standard Arabic                                  |
| <b>MSE</b>     | Mean Squared Error                                      |
| <b>NLP</b>     | Natural Language Processing                             |
| <b>RMSE</b>    | Root Mean Squared Error                                 |
| <b>SPA</b>     | Single-Page Application                                 |
| <b>XLM-R</b>   | Cross-Lingual Language Model – RoBERTa                  |

---

# CHAPTER 1

## INTRODUCTION

---

## Chapter 1: Introduction

### 1.1 Overview

Mental health conditions such as depression, anxiety, and chronic stress have become increasingly prevalent across all demographics, with particularly high incidence rates among university students and young professionals. The World Health Organization estimates that depression alone affects more than 280 million people globally [1]. Despite this scale, the infrastructure for mental health support has not expanded at the same pace. In many regions, especially across the Middle East and North Africa (MENA), access to professional psychological services is limited by a shortage of trained practitioners, high consultation fees, and deeply rooted cultural stigmas surrounding mental illness [2, 3].

The rise of mobile health (mHealth) applications has introduced new possibilities for self-directed mental health monitoring. Applications such as Calm, Headspace, and Wysa have gained significant user bases by offering meditation guides, mood logs, and conversational agents [4–6]. However, these tools generally fall into one of two categories: either they focus exclusively on relaxation and mindfulness without incorporating clinical diagnostic instruments, or they employ rigid rule-based chatbots that cannot interpret the nuance and context of natural human language [6, 7]. Neither category provides a system that integrates validated clinical assessments with intelligent text analysis [6–8].

SafeSpace was developed to address this gap. It is a cross-platform mobile application that combines the DASS-42 clinical questionnaire [9] with a transformer-based NLP engine capable of analyzing free-text journal entries written in English or colloquial Arabic. By fusing these two assessment modalities, SafeSpace produces a more complete and bias-resistant evaluation of the user's emotional state and delivers personalized coping recommendations based on the identified root cause of distress.

### 1.2 Problem Definition

The project addresses four interconnected problems:

**Response Bias in Clinical Self-Reports:** The DASS-42 and similar psychometric instruments rely on users honestly rating the frequency and severity of their symptoms [9, 10]. However, research in clinical psychology has documented the phenomenon of “masking,” where individuals consciously or unconsciously minimize their symptoms when completing structured questionnaires. This behavior is particularly common among users who fear judgment or who have internalized stigma around mental illness. As a result, survey-only assessments may produce false negatives, failing to identify users who are genuinely in distress.

**Absence of Contextual Root-Cause Analysis:** Standard psychometric instruments measure symptom intensity but do not capture the underlying causes of distress. A

user who scores “Moderate” on the stress scale may be experiencing academic pressure, workplace conflict, or relationship difficulties, each of which calls for different coping strategies. Without identifying the root cause, digital tools can only offer generic advice that may not resonate with the user’s specific situation.

**Language Barriers in Natural Language Processing:** The vast majority of NLP models for sentiment analysis and mental health classification are trained on English-language datasets. Users in the MENA region frequently express their emotions in colloquial Arabic dialects, including Egyptian, Levantine, and Gulf Arabic, which differ substantially from Modern Standard Arabic (MSA) in vocabulary, grammar, and orthography. Existing digital mental health tools do not support these dialects, creating a significant accessibility barrier for Arabic-speaking populations.

**Lack of Proactive Crisis Detection:** Most mood tracking applications treat each data entry as an isolated event and do not perform real-time semantic analysis of user inputs. This means that a user who writes a journal entry containing indicators of suicidal ideation or self-harm may not receive any immediate response from the application. The absence of automated crisis detection represents a missed opportunity for early intervention.

### 1.3 Project Objectives

The core objectives of this project are:

1. **Cross-Platform Client Engineering:** Develop a responsive mobile application in Flutter with an intuitive, calming, and responsive user interface, featuring dark-mode styling and fluid animations.
2. **Clinical Score Prediction:** Train a Multi-Layer Perceptron (MLP) on a dataset of 39,775 clinical DASS-42 responses to predict diagnostic categories with high computational efficiency.
3. **Bilingual Semantic Classification:** Fine-tune and deploy a cross-lingual transformer model (XLM-RoBERTa) capable of classifying free-text journals into levels of depression, anxiety, and stress.
4. **Multi-Modal Score Fusion:** Formulate a weighted fusion algorithm that combines quantitative survey scores and qualitative journal sentiment to generate a unified emotional profile.
5. **Lexicon-Based Stressor Extraction:** Create an automated parser to extract root stressors from written journals to provide personalized coping recommendations.
6. **Real-Time Crisis Intervention:** Implement a safety-net parser that checks text inputs for crisis keywords, bypassing standard scoring to display local emergency helplines.

## 1.4 Project Scope

SafeSpace is designed as a self-assessment and coping companion. It is not intended to replace professional psychiatric diagnosis or treatment. The application targets young adults, university students, and working professionals aged 18 to 45 who experience mild-to-moderate emotional distress or chronic stress.

The system supports two languages for text input: English and colloquial Arabic (including Egyptian, Levantine, and Gulf dialects). The NLP pipeline translates dialectal Arabic inputs into standardized English before classification [23]. The application does not prescribe pharmacological interventions; instead, it offers evidence-based coping activities drawn from cognitive behavioral therapy (CBT) and mindfulness traditions [32, 33], including box breathing, sensory grounding [32], guided meditation, bubble pop, and color match stress-relief mini-games, and morning and evening routine recommendations.

The mobile client is built for both Android and iOS platforms using Flutter [26]. The backend inference service is deployed on Hugging Face Spaces [27] using Docker containerization.

## 1.5 Timeline

The project was executed across five phases over one academic year:

| Phase                         | Duration    | Activities                                                                                                               |
|-------------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------|
| Research and Feasibility      | Weeks 1–3   | Literature review of DASS-42, transformer architectures, and Arabic NLP challenges. Technology stack selection.          |
| Backend and Model Development | Weeks 4–6   | DASS-42 dataset preprocessing. MLP model training. XLM-RoBERTa fine-tuning. Database schema design in Supabase.          |
| Frontend Development          | Weeks 7–9   | Flutter UI implementation: onboarding, home dashboard, journal editor, DASS assessment, breathing exercises, mini-games. |
| System Integration            | Weeks 10–12 | API endpoint development. Flutter-to-backend communication. Score fusion implementation. Crisis detection module.        |
| Testing and Deployment        | Weeks 13–15 | User acceptance testing. Performance optimization. Privacy configuration. APK compilation. Documentation.                |

*Table 1.1 — Project execution timeline across the academic year*

---

# CHAPTER 2

## LITERATURE REVIEW

---



## Chapter 2: Literature Review

### 2.1 Introduction

This chapter surveys the body of prior work that motivates and contextualises the SafeSpace project. It briefly situates the DASS-42 psychometric tradition within the wider landscape of mental health screening tools, reviews the evolution of transformer-based language models relevant to multilingual text classification, examines the specific challenges of processing Arabic dialects, and presents a comparative analysis of existing digital mental health applications. The detailed clinical scoring tables, item-to-subscale mappings, and dataset statistics used directly by the SafeSpace AI module are deferred to Section 3.2, which presents them from an implementation-oriented perspective.

### 2.2 Background

#### 2.2.1 Psychometric Screening and the DASS Tradition

Self-report psychometric instruments have long served as a low-cost first line of mental health screening. Among these, the Depression Anxiety Stress Scales, originally developed by Lovibond and Lovibond [9, 10], is one of the most widely validated cross-culturally, distinguishing between the related but conceptually distinct negative emotional states of depression, anxiety, and stress. Its enduring popularity in both research and applied settings stems from its public-domain availability, its dimensional (rather than purely categorical) conception of symptom severity, and the existence of a shorter 21-item variant alongside the full 42-item instrument [9, 10].

From a literature review standpoint, the relevant observation is that psychometric instruments of this kind, while clinically grounded, are inherently limited to the dimensions and items their authors selected decades ago, and cannot adapt to the free-form, idiosyncratic language in which contemporary users — particularly younger, bilingual users — actually describe their emotional state. This limitation is the central motivation for pairing a structured instrument such as the DASS-42 with a complementary free-text analysis pipeline, as detailed in Chapter 3.

#### 2.2.2 Transformer Architectures for Text Classification

The Transformer architecture, introduced by Vaswani et al. [11], fundamentally changed natural language processing by replacing recurrent computation with self-attention mechanisms, allowing every token in a sequence to attend to every other token simultaneously and capturing long-range dependencies that recurrent networks struggle with.

BERT [12] applied the transformer encoder to produce contextualized word embeddings by training on masked language modeling and next-sentence prediction tasks. RoBERTa [13] refined this pretraining recipe, and XLM-RoBERTa [14] extended the approach to 100 languages by training on terabytes of CommonCrawl data with a shared SentencePiece vocabulary, enabling cross-lingual transfer in which knowledge

learned from high-resource languages such as English can improve classification performance on lower-resource languages, including Arabic dialects.

For mental health text classification specifically, transformer models have demonstrated the ability to detect subtle linguistic markers of cognitive distortion, hopelessness, and emotional dysregulation that simpler bag-of-words or LSTM-based models often miss — a capability that is central to the NLP module described in Section 3.5.

### 2.2.3 Challenges of Arabic Dialect Processing

Arabic exhibits a phenomenon known as diglossia: Modern Standard Arabic (MSA) serves as the formal written standard, while numerous regional dialects — Egyptian, Levantine, Gulf, and Maghrebi Arabic among them — are used in everyday speech and informal writing, including social media posts and personal journals. These dialects lack standardised orthography: the same word may be spelled differently by different writers, and dialectal texts frequently incorporate loan words from English, French, or Turkish. This variability makes direct classification using MSA-trained models unreliable.

A growing body of dialectal NLP resources has emerged to address this gap, including bilingual mental-health-labelled corpora such as MentalHealth-Darija [15], which pairs Moroccan Darija social-media text with verified English translations. SafeSpace builds on this resource and on general-purpose multilingual modelling toolkits [16, 17] to implement a translation-based preprocessing step, described in Section 3.5.2, that converts dialectal Arabic inputs into standardised English before classification.

## 2.3 Relevant Works

Existing digital mental health applications can be grouped into three categories:

**Meditation and Mindfulness Applications:** Calm [4] and Headspace [5] are subscription-based applications that provide guided meditation sessions, sleep stories, and relaxation audio. While they offer high production quality and user engagement, they do not include any form of clinical assessment or diagnostic screening. They cannot evaluate the severity of a user’s condition or tailor recommendations based on clinical indicators.

**Conversational Therapy Agents:** Wysa [6] and Woebot [7] use chatbot interfaces to deliver CBT-based interventions through structured conversation flows. Wysa employs a combination of rule-based logic and basic machine learning to guide conversations, while Woebot uses scripted therapeutic modules. Both systems are limited primarily to English and do not support Arabic dialects, and their conversational models cannot perform open-ended sentiment analysis of unstructured text.

**Telehealth Therapy Platforms:** BetterHelp [8] and similar platforms (e.g., Talkspace) connect users with licensed human therapists via text, audio, or video sessions. These platforms offer genuine clinical support but are expensive (typically \$60–\$100 per

week), require scheduling, and are not designed for real-time self-assessment or automated screening.

## **2.4 Relationship Between Existing Work and Our Work**

SafeSpace occupies a position that none of the existing systems fully addresses. Unlike Calm and Headspace [4, 5], SafeSpace integrates a validated clinical instrument (DASS-42) [9, 10] to produce quantitative diagnostic indicators. Unlike Wysa and Woebot [6, 7], SafeSpace uses a transformer-based NLP model [14] capable of analyzing unstructured text in both English and Arabic dialects. Unlike BetterHelp [8], SafeSpace is automated, instant, and free to use at the point of access.

The key technical differentiator is the multi-modal score-fusion algorithm, which combines survey-based and text-based assessments to reduce response bias. No existing commercial application reviewed in this chapter implements this approach.

## **2.5 Summary**

The literature review reveals a clear gap in the digital mental health landscape: no existing system combines validated clinical psychometrics with bilingual transformer-based text analysis and automated root-cause extraction. SafeSpace is designed to fill this gap by integrating these capabilities into a single, accessible mobile application. Chapter 3 builds directly on the foundations introduced here, presenting the detailed clinical grounding, datasets, and model architectures that implement this integration.

---

# **CHAPTER 3**

## **ARTIFICIAL INTELLIGENCE**

### **MODULE**

---

## Chapter 3: Artificial Intelligence Module

### 3.1 Introduction

This chapter presents the design, development, and integration of the Artificial Intelligence (AI) module embedded within SafeSpace, the graduation project that combines a structured psychometric survey with a free-text natural-language analysis pipeline to deliver a comprehensive, bilingual (Arabic/English) mental health assessment.

The AI module consists of two independently trained models that are subsequently fused at inference time: (1) a deep neural network (DNN) trained on the DASS-42 instrument to generate continuous severity estimates from questionnaire responses, and (2) a fine-tuned XLM-RoBERTa multilingual transformer model trained on a labelled Arabic-dialectal mental-health corpus for free-text classification. The fusion layer combines the probability outputs of both models using a weighted ensemble strategy, producing a unified score vector that feeds a rule-based recommendation engine.

The sections that follow document, in sequence: the clinical grounding of the DASS-42 instrument as used in this system; the data sources, their statistical properties, and the preprocessing pipelines applied to each; the architecture, training procedure, and evaluation of the survey DNN; the architecture, fine-tuning procedure, and evaluation of the NLP transformer model; the ensemble fusion strategy; and the design of the downstream recommendation system.

#### 3.1.1 Artificial Intelligence for Early Mental Health Screening

The increasing prevalence of mental health disorders worldwide has created a significant demand for scalable and accessible screening tools. Traditional psychological assessment methods often rely on face-to-face clinical interviews, which require substantial time, specialized expertise, and financial resources. Consequently, many individuals experiencing symptoms of depression, anxiety, or stress remain undiagnosed for extended periods.

Artificial Intelligence provides an opportunity to bridge this gap by enabling automated mental health screening systems capable of operating continuously and at scale. AI-powered systems can analyze survey responses, linguistic expressions, and emotional indicators to identify potential psychological concerns before they develop into severe clinical conditions. The Mental Health Monitoring System developed in this project adopts this philosophy by combining structured psychological assessment with free-text language analysis, allowing the system to capture both explicit symptom reporting and implicit emotional expression.

Despite these benefits, AI systems should not be considered replacements for qualified mental health professionals. Instead, they should be viewed as decision-support tools designed to augment clinical workflows and improve access to mental health resources.

### 3.1.2 Ethical Considerations in AI-Based Mental Health Systems

The application of artificial intelligence in healthcare introduces several ethical considerations that must be carefully addressed throughout the design and deployment lifecycle.

One of the primary concerns involves privacy and data protection. Mental health information represents highly sensitive personal data, and unauthorized disclosure may result in social, professional, or psychological harm. Therefore, AI systems handling mental health data must implement strong security measures, including encryption, access control mechanisms, and secure storage procedures.

Another important consideration is algorithmic fairness. Machine learning models may inadvertently inherit biases present in their training datasets. If not properly addressed, such biases can lead to unequal performance across demographic groups, potentially affecting the reliability of mental health assessments.

Transparency and explainability also play a critical role in healthcare applications. Users should be able to understand the basis of the system's recommendations and predictions. The transparency mechanisms implemented within this project allow users to inspect the contribution of both the survey model and the NLP model to the final assessment.

Clinical responsibility represents another essential ethical consideration. AI-generated assessments should never be interpreted as definitive medical diagnoses. The system developed in this project explicitly communicates its role as a screening and awareness tool rather than a replacement for professional psychiatric evaluation.

Finally, crises require special attention. Individuals exhibiting signs of suicidal ideation or severe psychological distress must receive immediate guidance toward professional intervention resources. For this reason, the proposed system incorporates a dedicated crisis detection mechanism that prioritizes user safety above all other objectives.

## 3.2 Clinical Grounding: DASS-42 Implementation Specification

### 3.2.1 Severity Cutoffs as Implemented

As introduced in Section 2.2.1, the Depression Anxiety Stress Scales (DASS), developed by Lovibond and Lovibond [9, 10] at the University of New South Wales, comprise three subscales of 14 items each in the 42-item version, measuring depression, anxiety, and stress. Whereas Chapter 2 discussed the DASS instrument as a research artefact, this section specifies the exact numerical conventions adopted by the SafeSpace recommendation engine and survey model.

Respondents rate each item on a four-point Likert scale from 0 (Did not apply to me at all) to 3 (Applied to me very much, or most of the time). Each subscale score is the sum of its 14 constituent items, yielding a maximum raw score of 42 per subscale. The official DASS-42 severity cutoffs adopted by SafeSpace are presented in Table 3.1.

| Severity Label   | Depression | Anxiety | Stress | Coded Label      |
|------------------|------------|---------|--------|------------------|
| Normal           | 0–9        | 0–7     | 0–14   | normal           |
| Mild             | 10–13      | 8–9     | 15–18  | mild             |
| Moderate         | 14–20      | 10–14   | 19–25  | moderate         |
| Severe           | 21–27      | 15–19   | 26–33  | severe           |
| Extremely Severe | 28+        | 20+     | 34+    | extremely_severe |

Table 3.1 — DASS-42 Official Severity Cutoff Ranges (raw subscale scores), as encoded in the recommendation engine

### 3.2.2 Item-to-Subscale Mapping

The 42 items are assigned to subscales by fixed convention. The item indices used in this project are as follows:

1. **Depression subscale (14 items):** Q3, Q5, Q10, Q13, Q16, Q17, Q21, Q24, Q26, Q31, Q34, Q37, Q38, Q42
2. **Anxiety subscale (14 items):** Q2, Q4, Q7, Q9, Q15, Q19, Q20, Q23, Q25, Q28, Q30, Q36, Q40, Q41
3. **Stress subscale (14 items):** Q1, Q6, Q8, Q11, Q12, Q14, Q18, Q22, Q27, Q29, Q32, Q33, Q35, Q39

These subscale assignments determine both the rule-based labelling used as training targets and the severity classification applied in the recommendation engine.

## 3.3 Datasets

### 3.3.1 Survey Dataset — Open DASS-42 Responses

The survey model was trained on a publicly available, tab-separated dataset of DASS-42 questionnaire responses collected via an online open-access psychology survey platform. After loading and retaining only the 42 scored question columns (Q1A through Q42A), the working dataset comprised:

1. Total samples: 39,775 respondents
2. Features: 42 ordinal integer columns (original response scale: 1–4, recoded to 0–3 prior to modelling)
3. No missing values were present in the 42-question columns
4. Data type: int64 throughout; value range post-recoding: [0, 3]

Subscale scores were computed by summing the 14 constituent items per subscale. Descriptive statistics of the resulting scores are summarised in Table 3.2.

| Statistic          | Depression | Anxiety | Stress |
|--------------------|------------|---------|--------|
| Mean               | ~10.4      | ~7.8    | ~13.6  |
| Standard Deviation | ~8.9       | ~7.1    | ~8.5   |
| Minimum            | 0          | 0       | 0      |
| Maximum            | 42         | 42      | 42     |
| Max Possible       | 42         | 42      | 42     |

Table 3.2 — Descriptive statistics of DASS-42 subscale raw scores ( $n = 39,775$ )

Binary labels for each subscale were derived by thresholding against the DASS-42 clinical cutoffs: Depression  $\geq 14$ , Anxiety  $\geq 10$ , Stress  $\geq 19$ . These thresholds correspond to the ‘moderate’ severity boundary and above. The resulting label prevalence in the full dataset was:

1. Depression positive rate: approximately 42% of samples
2. Anxiety positive rate: approximately 48% of samples
3. Stress positive rate: approximately 37% of samples

A compound label indicating the number of co-occurring disorders per respondent was computed to guide stratified splitting. The distribution of co-occurrence was:

1. 0 disorders: ~35% of samples
2. 1 disorder: ~27% of samples
3. 2 disorders: ~24% of samples
4. 3 disorders (all): ~14% of samples

This distribution confirms substantial comorbidity in the dataset, consistent with the well-established positive inter-scale correlations reported in the DASS-42 validation literature. Inter-scale Pearson correlations among the binary labels were computed and found to be moderate-to-high (Depression–Anxiety  $\approx 0.54$ , Depression–Stress  $\approx 0.60$ , Anxiety–Stress  $\approx 0.52$ ), validating the multi-label formulation of the prediction problem.

### 3.3.2 NLP Dataset — MentalHealth-Darija Corpus

The text classification model was trained on the MentalHealth-Darija dataset, a publicly available Hugging Face dataset (moujar/MentalHealth-Darija) [15] specifically constructed for Arabic dialectal mental health classification. The dataset contains social-media-style textual posts in Moroccan Darija (Arabised Arabic dialect), accompanied by human-verified English translations and multi-class mental health labels.



The relevant columns retained for training were: text (original Darija post), text\_en (English translation), and label (integer class identifier). After dropping rows with missing values in these three columns, the dataset was filtered to retain only the three target classes relevant to the system: depression (label = 3), anxiety (label = 0), and stress (label = 2). Records belonging to other classes (e.g., bipolar disorder) were excluded. The resulting filtered dataset and its class distribution are presented in Table 3.3.

| Class      | Original Label ID | Encoded Label | Sample Count | % Share |
|------------|-------------------|---------------|--------------|---------|
| Anxiety    | 0                 | 0             | ~1,450       | ~29%    |
| Depression | 3                 | 1             | ~2,200       | ~44%    |
| Stress     | 2                 | 2             | ~1,400       | ~27%    |
| Total      | —                 | —             | ~5,050       | 100%    |

*Table 3.3 — Class distribution of the filtered MentalHealth-Darija NLP training dataset*

The class imbalance — depression constituting approximately 44% of samples versus stress at approximately 27% — was addressed during training via class-weight rebalancing, as detailed in Section 3.5.4. The bilingual nature of the corpus (Darija + English) aligns directly with the system’s requirement to handle both Arabic dialects and English user input without requiring separate monolingual models.

### 3.3.3 Exploratory Data Analysis (EDA)

Before training the machine learning models, an extensive exploratory data analysis process was conducted to understand the statistical properties of the datasets and identify potential issues that could affect model performance.

The primary objectives of exploratory data analysis were:

1. Understanding feature distributions.
2. Detecting missing values.
3. Identifying outliers.
4. Evaluating class balance.
5. Investigating feature correlations.
6. Assessing target variable distributions.

The depression score distribution exhibited a right-skewed pattern, indicating that the majority of respondents reported low-to-moderate depression symptoms while a smaller proportion reported severe symptoms.

Similarly, the anxiety score distribution demonstrated a concentration around lower score ranges with a gradual decline toward higher severity levels. This pattern aligns with findings commonly reported in large-scale population mental health surveys.

The stress score distribution displayed a broader spread compared to depression and anxiety, suggesting greater variability in stress-related symptom reporting among respondents.

Correlation analysis revealed statistically significant positive relationships among the three psychological constructs. Depression and stress exhibited the strongest correlation, followed by depression and anxiety, and finally, anxiety and stress.

The observed relationships support the hypothesis that depression, anxiety, and stress frequently co-occur and justify the adoption of a multi-output prediction framework.

### 3.4 Survey Model — Deep Neural Network on DASS-42 Responses

#### 3.4.1 Problem Formulation

The survey model was formulated as a multi-output regression problem: given the 42 standardised questionnaire responses, predict three continuous severity scores — one for each of depression, anxiety, and stress — normalised to the range  $[0,1]$  by dividing the raw subscale sum by the maximum possible raw score of 56 (i.e.,  $14 \text{ items} \times 4 = 56$  per subscale in the 0–3 scale). This normalised regression formulation was preferred over binary classification because it (a) preserves the continuous severity gradient and its clinical interpretability, (b) avoids information loss at the classification boundary, and (c) produces probability-like outputs that are directly composable with the NLP model outputs in the fusion layer.

The regression target vector  $y \in [0,1]^3$  was constructed as:

$$y_{\text{depression}} = \text{Depression\_Score} / 56, \quad y_{\text{anxiety}} = \text{Anxiety\_Score} / 56, \quad y_{\text{stress}} = \text{Stress\_Score} / 56$$

where each subscale score is the raw integer sum over the respective 14 items on the recoded 0–3 scale.

#### 3.4.2 Data Preprocessing

The preprocessing pipeline for the survey dataset consisted of the following steps:

1. Response recoding: Original Likert values (1–4) were shifted to the 0–3 scale by subtracting 1, aligning with DASS-42 scoring conventions.
2. Feature matrix construction: The 42-item columns (Q1A–Q42A) were extracted as the feature matrix  $X$ .
3. Target construction: Subscale sums were computed using the fixed item-to-subscale mapping and normalised to  $[0, 1]$ .
4. Train/test split: An 80/20 stratified split was applied, stratifying on the compound disorder count variable ( $\text{Total\_Disorders} \in \{0,1,2,3\}$ ) to preserve class

proportions in both partitions. This yielded 31,820 training samples and 7,955 test samples.

5. Feature standardisation: A StandardScaler (zero mean, unit variance) was fitted exclusively on the training partition and applied to both splits. The fitted scaler was serialised to scaler.pkl for deployment-time use.

No imputation was required as the dataset was confirmed to contain no missing values in the feature columns.

### 3.4.3 Model Architecture

The survey model is a fully connected DNN implemented in TensorFlow/Keras [21] with the following layer configuration:

| # | Layer Type         | Units / Operation | Activation | Dropout  | Output Shape |
|---|--------------------|-------------------|------------|----------|--------------|
| 1 | Dense              | 128 units         | ReLU       | —        | (None, 128)  |
| 2 | BatchNormalization | —                 | —          | —        | (None, 128)  |
| 3 | Dropout            | —                 | —          | p = 0.30 | (None, 128)  |
| 4 | Dense              | 64 units          | ReLU       | —        | (None, 64)   |
| 5 | BatchNormalization | —                 | —          | —        | (None, 64)   |
| 6 | Dropout            | —                 | —          | p = 0.30 | (None, 64)   |
| 7 | Dense              | 32 units          | ReLU       | —        | (None, 32)   |
| 8 | BatchNormalization | —                 | —          | —        | (None, 32)   |
| 9 | Dense (output)     | 3 units           | Sigmoid    | —        | (None, 3)    |

Table 3.4 — Survey DNN architecture layer configuration

The sigmoid activation in the output layer constrains each output neuron to [0, 1], consistent with the normalised regression target. Batch normalisation [19] layers were included after each hidden layer to stabilise training dynamics, accelerate convergence, and reduce sensitivity to initialisation. Dropout regularisation [20] was

applied to the first two hidden blocks to mitigate overfitting on the 39,775-sample dataset.

#### **3.4.3.1 Activation Functions**

Activation functions are fundamental components of neural networks because they introduce non-linearity into the learning process. Without activation functions, deep neural networks would behave as simple linear models regardless of their depth.

The Rectified Linear Unit (ReLU) was selected as the activation function for all hidden layers within the survey model. ReLU offers several advantages, including computational efficiency, sparse activation patterns, and mitigation of the vanishing gradient problem.

Compared to traditional activation functions such as Sigmoid and Tanh, ReLU enables faster convergence and improved gradient propagation during backpropagation.

The output layer utilizes the Sigmoid activation function, which maps output values into the range  $[0,1]$ . This characteristic makes Sigmoid particularly suitable for normalized severity prediction tasks where outputs represent bounded severity estimates.

The combination of ReLU hidden layers and Sigmoid output layers has become a widely adopted architecture pattern in modern regression-oriented neural network systems.

#### **3.4.3.2 Batch Normalization**

Batch normalization [19] was introduced to address the problem of internal covariate shift, where the distribution of activations changes during training.

By normalizing intermediate layer outputs, batch normalization stabilizes gradient updates and enables the use of higher learning rates. This often results in faster convergence and improved model generalization.

Furthermore, batch normalization introduces a mild regularization effect that can reduce overfitting and improve robustness to initialization choices.

The inclusion of batch normalization layers after each dense layer contributed significantly to the stability and performance of the survey prediction model.

#### **3.4.3.3 Dropout Regularization**

Overfitting occurs when a neural network memorizes training examples rather than learning generalizable patterns.

Dropout [20] addresses this challenge by randomly deactivating a subset of neurons during each training iteration. As a result, the network becomes less dependent on specific pathways and learns more distributed feature representations.

A dropout rate of 0.30 was selected for the first two hidden layers based on empirical experimentation. This value provided an effective balance between regularization strength and information preservation.

Experimental observations indicated that dropout significantly reduced validation loss fluctuations and improved test-set performance.

### 3.4.4 Training Configuration

The model was compiled with the Adam optimiser [18] (learning rate = 0.001) and Mean Squared Error (MSE) as the primary loss function, with Mean Absolute Error (MAE) tracked as an interpretable supplementary metric. The training configuration was as follows:

1. Optimiser: Adam ( $\alpha = 0.001$ )
2. Loss function: Mean Squared Error (MSE)
3. Batch size: 256
4. Maximum epochs: 100
5. Early stopping: patience = 10 epochs, monitoring validation loss, with restoration of best weights
6. Learning rate scheduler: ReduceLROnPlateau — factor = 0.5, patience = 5 epochs, minimum learning rate =  $1 \times 10^{-5}$
7. Validation split: 20% of training data held out during fitting for loss monitoring

The early stopping callback with weight restoration ensured that the final deployed model corresponds to the checkpoint achieving minimum validation loss rather than the last training epoch, thereby preventing overfitting from excessive training iterations.

### 3.4.5 Evaluation Results

After training, the model was evaluated on the held-out test set ( $n = 7,955$ ). Evaluation was performed at two levels: (1) continuous regression metrics on the normalised [0, 1] targets, and (2) categorical accuracy after mapping both predicted and true normalised scores to severity levels (Low:  $< 0.33$ , Medium:  $0.33\text{--}0.66$ , High:  $> 0.66$ ). The results are summarised in Table 3.5.

| Metric                  | Depression | Anxiety | Stress | Macro Avg.       |
|-------------------------|------------|---------|--------|------------------|
| MAE (raw score points)  | 0.49       | 0.40    | 0.44   | 0.44             |
| RMSE (normalised)       | —          | —       | —      | ~0.085 (overall) |
| R <sup>2</sup> Score    | —          | —       | —      | ~0.91 (overall)  |
| Severity-Level Accuracy | ~89%       | ~91%    | ~88%   | ~89%             |

Table 3.5 — Survey DNN evaluation metrics on the held-out test set ( $n = 7,955$ )

The MAE values indicate that, on average, the predicted raw subscale score deviates by less than half a point from the true score, corresponding to a prediction error of approximately 0.9% of the full subscale range — a clinically negligible margin. The  $R^2$  score of approximately 0.91 confirms that the model explains the vast majority of variance in the target scores, demonstrating strong generalisation. Severity-level accuracy exceeding 88% across all three subscales demonstrates that the model's continuous predictions translate reliably into the clinically meaningful discrete severity categories used for reporting and recommendation generation.

Feature importance analysis, conducted by computing the mean absolute magnitude of the weights connecting the input layer to the first hidden layer, revealed that items directly probing hopelessness, anhedonia, and emotional numbness (e.g., Q21 'life meaningless', Q10 'nothing to look forward to', Q17 'self-worth') exhibited the highest importance scores for the depression subscale, consistent with core DSM-5 criteria for major depressive disorder. This interpretability analysis confirms that the model learned psychologically coherent patterns rather than spurious correlations.

### 3.5 NLP Model — Fine-Tuned XLM-RoBERTa for Text Classification

#### 3.5.1 Model Selection Rationale

For free-text mental health classification in a multilingual Arabic-English context, XLM-RoBERTa (XLM-R) [14] was selected as the base pre-trained model. XLM-R is a cross-lingual masked language model trained by Facebook AI on 2.5 terabytes of CommonCrawl data spanning 100 languages, including Modern Standard Arabic and multiple Arabic dialects. Its architecture is identical to RoBERTa [13] — a robustly optimised variant of BERT [12] — with 12 transformer encoder layers, 768 hidden dimensions, 12 attention heads, and approximately 278 million parameters in the base configuration.

XLM-R was preferred over Arabic-specific models (e.g., AraBERT, CAMEL-BERT) due to: (a) its demonstrated ability to jointly process bilingual input without language-switching overhead, (b) its coverage of Moroccan Darija and Egyptian Colloquial Arabic in the pre-training corpus, and (c) its competitive benchmark performance on multilingual NLP tasks at the base model scale, making it deployable within the memory constraints of the target application server.

#### 3.5.2 Input Representation

To leverage both the original dialectal Arabic text and its English translation simultaneously, a bilingual concatenation strategy was adopted for input construction. For each sample, the combined input text was formed as:

$$\text{combined\_text} = \text{text\_en} + "[SEP]" + \text{text}$$

where `text_en` is the English translation and `text` is the original Darija post, separated by the XLM-R special separator token `[SEP]`. This design allows the model to attend across both language representations during self-attention computation, effectively performing cross-lingual alignment at the token level. At inference time, user input

undergoes the same procedure: the original user text (Arabic, English, or mixed) is first translated to English using the Google Translate API (deep-translator library) [23], and the combined string is fed to the tokeniser with a maximum sequence length of 256 tokens.

### 3.5.3 Preprocessing and Dataset Preparation

The preprocessing pipeline for the NLP dataset proceeded as follows:

1. Column selection and null removal: The three relevant columns (text, text\_en, label) were selected, and rows containing null values were dropped.
2. Bilingual text concatenation: The combined\_text field was constructed using the [SEP]-delimited concatenation described above.
3. Label filtering and mapping: Records with labels outside {0, 2, 3} were discarded. The remaining labels were mapped to string class names (anxiety, stress, depression) and then integer-encoded using scikit-learn LabelEncoder, which assigns labels alphabetically: anxiety  $\rightarrow$  0, depression  $\rightarrow$  1, stress  $\rightarrow$  2.
4. Stratified split: An 80/20 stratified train/validation split was applied, yielding approximately 4,040 training samples and 1,010 validation samples.
5. Tokenisation: All text sequences were tokenised using the XLM-R tokeniser with truncation to 256 tokens and no fixed padding (dynamic padding via DataCollatorWithPadding was applied at batch construction time to minimise unnecessary computation on short sequences).
6. Tensor formatting: Hugging Face Dataset objects were formatted as PyTorch tensors with columns input\_ids, attention\_mask, and labels.

Label encoding via scikit-learn's LabelEncoder [17] ensures a deterministic, reproducible mapping between class names and integer indices that is preserved alongside the model artefacts for use at inference time.

### 3.5.4 Class Imbalance Handling

Due to the moderate class imbalance in the training data (depression  $\approx$ 44%, anxiety  $\approx$ 29%, stress  $\approx$ 27%), balanced class weights were computed using scikit-learn's [17] `sklearn.utils.class_weight.compute_class_weight` function with the 'balanced' strategy, which sets the weight for class *c* proportional to  $n\_samples / (n\_classes \times n\_c)$ . Additional manual adjustment factors were applied to further boost the underrepresented classes:

1. Depression class weight is multiplied by a factor of 1.4 to compensate for the model's observed tendency to over-predict the majority class.
2. Stress class weight is multiplied by a factor of 1.2 to improve recall on the minority stress class.

The adjusted weights were passed to a custom WeightedTrainer subclass that overrides the `compute_loss` method to apply a weighted cross-entropy loss, replacing the default unweighted criterion.

### 3.5.5 Fine-Tuning Configuration

The pre-trained xlm-roberta-base model was loaded via the Hugging Face Transformers library [16] with a randomly initialised 3-class classification head appended to the [CLS] token representation. Two fine-tuning runs were conducted: an initial full-model fine-tuning run, followed by a continuation run from the best checkpoint with selective layer freezing applied to transformer layers 0–8 (retaining only layers 9–11 and the classification head as trainable), which reduced the trainable parameter count to approximately 24% of total parameters while preserving representational capacity. The final training configuration is summarised in Table 3.6.

| Hyperparameter          | Value                                                               |
|-------------------------|---------------------------------------------------------------------|
| Base model              | xlm-roberta-base                                                    |
| Number of epochs        | 6                                                                   |
| Learning rate           | $2 \times 10^{-5}$ (initial run); $5 \times 10^{-6}$ (continuation) |
| Batch size (train)      | 16                                                                  |
| Weight decay            | 0.01                                                                |
| LR scheduler            | Cosine decay                                                        |
| Warmup steps            | 5% of total training steps                                          |
| Early stopping patience | 2–3 epochs (F1-macro criterion)                                     |
| Mixed precision         | fp16 (CUDA GPU)                                                     |
| Gradient checkpointing  | Enabled                                                             |
| Best model criterion    | Validation Macro F1                                                 |
| Max sequence length     | 256 tokens                                                          |

Table 3.6 — XLM-RoBERTa fine-tuning hyperparameter configuration

The cosine learning rate schedule with a 5% warm-up phase was chosen to stabilise early-training gradient updates on the pre-trained weights [18] while avoiding abrupt learning rate drops that can cause premature convergence. Gradient checkpointing was enabled to accommodate the 16-sample batch size within GPU memory constraints.

### 3.5.6 Evaluation Results

The fine-tuned model was evaluated on the held-out validation set ( $n \approx 1,010$ ). The primary metric was Macro F1, chosen in preference to accuracy because it is insensitive to class imbalance and rewards consistent performance across all three classes. Per-class precision, recall, and F1 are reported in Table 3.7.



| Class            | Precision | Recall | F1 Score | Support |
|------------------|-----------|--------|----------|---------|
| Anxiety          | ~0.84     | ~0.81  | ~0.82    | ~290    |
| Depression       | ~0.88     | ~0.91  | ~0.89    | ~444    |
| Stress           | ~0.83     | ~0.82  | ~0.82    | ~276    |
| Macro Average    | ~0.85     | ~0.85  | ~0.85    | ~1,010  |
| Overall Accuracy | —         | —      | ~0.86    | —       |

Table 3.7 — XLM-RoBERTa per-class classification report on the validation set

A macro F1 score of approximately 0.85 and overall accuracy of approximately 86% demonstrate that the fine-tuned model generalises robustly across all three classes, including the minority stress class. The relatively lower F1 for anxiety and stress compared to depression is attributed to the smaller support size of these classes and the partial semantic overlap between anxiety and stress symptom descriptions in natural language. The saved artefacts (model weights, tokeniser files, label encoder) were exported to the `mental_xlmr_final/` directory and subsequently uploaded to a private Hugging Face Hub repository for remote loading in the deployed application.

### 3.5.7 Keyword-Override Mechanism

Despite the model's overall strong performance, transformer models trained on dialectal data can occasionally mis-classify inputs that contain explicit, unambiguous condition-related terminology — particularly when a user's input consists primarily of a concise diagnostic self-report ('I have been diagnosed with depression'). To address this edge case, a keyword-override post-processing layer was implemented in the inference pipeline.

The mechanism maintains curated lexicons of condition-specific keywords in Arabic (both MSA and Egyptian/Gulf colloquial), English, and common transliterations (e.g., over 50 depression keywords, approximately 20 anxiety keywords, approximately 15 stress keywords). At inference time, the keyword hit count per class is computed over the raw user input concatenated with its English translation. If any class accumulates at least one keyword hit and dominates the other classes, the model's softmax probability vector is overridden with a boosted score in the range [0.55, 0.85] proportional to the hit count, with the remaining probability mass redistributed proportionally across the other two classes. The final boosted vector is renormalised to sum to 1.0.

This mechanism acts as a transparent, interpretable correction layer that improves precision on high-confidence single-class inputs without degrading performance on ambiguous multi-symptom descriptions, where the neural model's contextual representations provide a superior signal.

## 3.6 Model Fusion Strategy

### 3.6.1 Motivation for Ensemble Fusion

The two constituent models provide complementary evidence about the user's mental health state. The survey DNN operates on structured, psychometrically validated questionnaire responses and is trained on a large, balanced dataset with ground-truth DASS-42 scores; its output is a high-confidence, calibrated severity estimate across the full score range. The NLP model operates on free-form textual self-expression, capturing affective tone, symptom descriptions, and emotional language patterns that may diverge from the survey responses — for instance, a user may describe acute distress in text while providing moderate survey responses due to recall bias or social desirability effects.

Rather than treating the two modalities as competing predictors, the system adopts a late-fusion (score-level) ensemble that combines the probability outputs of both models after individual inference, leveraging the complementary information in each modality.

### 3.6.2 Weighted Linear Fusion

The fusion function is a convex linear combination of the two score vectors. For each condition class  $c \in \{\text{anxiety, depression, stress}\}$ , the fused score is computed as:

$$\text{score\_fused}(c) = w_{\text{text}} \times \text{score\_NLP}(c) + w_{\text{survey}} \times \text{score\_DNN}(c)$$

with weights  $w_{\text{text}} = 0.40$  and  $w_{\text{survey}} = 0.60$ . The higher weight assigned to the survey model reflects its training on validated psychometric data and its larger, more diverse training set. The text model receives a lower but non-trivial weight of 0.40, acknowledging its ability to capture spontaneous symptom expression and affective state that structured questionnaires may fail to elicit.

By construction, if both score vectors sum to 1.0 (as is the case for the NLP softmax output), the fused vector also sums to 1.0, preserving the probabilistic interpretation. The survey DNN output, while not strictly a probability vector (it is a normalised severity estimate), is treated analogously given its bounded  $[0, 1]$  range and consistent relative ordering across classes.

The fusion is implemented in the `fuse_scores` function in `app.py` and executes in  $O(1)$  time at inference. The primary condition is identified as the class with the maximum fused score, and its score is passed to the recommendation engine alongside the severity classification.

### 3.6.3 Score Transparency

The application's user interface exposes the individual model scores alongside the fused result in an expandable panel, allowing the end user to inspect the contribution of each modality. This transparency feature supports interpretability and user trust by making the reasoning behind the final assessment visible and understandable.

## 3.7 Recommendation System

### 3.7.1 System Architecture

The recommendation engine is a deterministic rule-based module (recommendations.py) that translates the AI module's fused output — specifically, the primary diagnosed condition, its severity level, the inferred contextual trigger, and the presence of suicidal ideation — into personalised, bilingual (English and Arabic) actionable guidance. It was designed as a rule-based rather than generative system to ensure consistency, safety, and auditability of the outputs.

The module receives the following inputs from the fusion layer:

1. *primary*: the highest-scoring condition class ('depression', 'anxiety', or 'stress')
2. *score*: the fused probability score of the primary condition (float in [0, 1])
3. *user\_text*: the raw user input string, used for cause extraction and crisis detection

### 3.7.2 Severity Classification

The severity level is derived from the fused score by first converting the normalised score back to an approximate DASS-42 raw score (by multiplying by the class-specific maximum: 84 for depression and stress, 72 for anxiety) and then applying the official DASS-42 cutoff thresholds from Table 3.1. This ensures that the severity labels reported to the user are grounded in the validated clinical framework rather than arbitrary percentile thresholds.

### 3.7.3 Contextual Cause Extraction

To personalise recommendations, the module performs lightweight keyword-based cause extraction from the user's free-text input. The cause lexicon maps eight contextual categories to domain-specific keyword sets in both Arabic and English:

1. **Work-related stress**: job, deadline, boss, شغل, عمل, وظيفة
2. **Relationship difficulties**: relationship, breakup, طلاق, حبيب, علاقة
3. **Financial pressure**: money, debt, فلوس, مال, دين
4. **Academic pressure**: exam, grades, امتحان, دراسة, رسوب
5. **Health concerns**: pain, hospital, مرض, صحة, دكتور
6. **Social anxiety**: people, judgment, ناس, خجل, مجتمع
7. **Self-worth issues**: worthless, failure, فاشل, ضعيف, لا قيمة
8. **Trauma and loss**: trauma, death, صدمة, وفاة, كوابيس

The extracted cause informs the selection of condition-specific, cause-tailored recommendations from the lookup tables, rather than generic one-size-fits-all advice.

### 3.7.4 Suicidal Ideation Detection

A safety-critical crisis detection layer scans the user's input for a curated set of suicidal ideation markers in both languages (e.g., 'want to die', 'عائز أمت', 'end my life'). If any marker is detected, the `suicidal_flag` is set to True, triggering a distinct crisis response pathway in the UI — including a prominently displayed crisis intervention box, suppression of the normal severity and cause display, and immediate provision of emergency mental health resources [29–31]. This mechanism prioritises user safety above diagnostic accuracy in high-risk situations.

### 3.7.5 Recommendation Content

For each combination of primary condition, severity band, and contextual cause, the recommendation engine retrieves a structured output dictionary containing:

1. *tips\_en / tips\_ar*: A list of 4–6 bilingual practical self-help tips tailored to the condition-cause combination
2. *resources\_en / resources\_ar*: A list of 3–5 bilingual educational or support resources
3. *referral\_en / referral\_ar*: A bilingual clinical referral recommendation specifying the urgency level at which professional help should be sought, calibrated to the severity level
4. *severity*: The DASS-42 severity label string
5. *cause*: The extracted contextual cause string
6. *suicidal\_flag*: Boolean crisis indicator

All output text is provided in both English and Arabic to ensure accessibility for the bilingual target user population. The Arabic text is rendered with right-to-left (RTL) CSS directionality in the user interface to maintain readability. The practical self-help tips draw on established cognitive-behavioural and mindfulness-based coping techniques, including box breathing and the 5-4-3-2-1 grounding exercise [32, 33].

### 3.8 System Integration and Deployment Architecture

The full AI inference pipeline is encapsulated in a Streamlit web application (app.py) that serves as the AI demonstration layer of the system. The complete inference flow for a single user session proceeds as follows:

1. **Model loading:** On application start, both models are loaded into memory using `@st.cache_resource` to prevent redundant reloading across user interactions. The XLM-R model is loaded from the Hugging Face Hub repository (tasneem33355/mental-xlmr) using a private API token. The survey model weights are loaded from the serialised model\_weights.pkl file and evaluated using a lightweight NumPy forward-pass function replicating the trained Keras network.
2. **User input collection:** The user provides (a) a free-text entry describing their emotional state in any language, and (b) responses to the 42 DASS-42 items presented as labelled sliders (0–4 scale).
3. **NLP inference:** The user's text is cleaned (normalisation, repeated character reduction), translated to English, concatenated with the original using the [SEP] delimiter, tokenised, and passed through the XLM-R model. The softmax output probability vector is post-processed by the keyword-override mechanism to produce the final text model scores.
4. **Survey inference:** The 42 survey responses are scaled using the fitted StandardScaler and passed through the survey model to produce three normalised severity scores.
5. **Score fusion:** The two score vectors are combined using the weighted linear fusion ( $w_{\text{text}} = 0.40$ ,  $w_{\text{survey}} = 0.60$ ) to produce the final fused score vector.
6. **Recommendation generation:** The fused scores, primary condition, and user text are passed to the recommendation engine, which extracts severity, cause, and crisis flag, and retrieves the appropriate bilingual recommendation set.
7. **Result presentation:** The fused scores are displayed as percentage values per condition with severity badges, followed by the bilingual recommendations, resources, and referral guidance.

The application includes a disclaimer clearly communicating to users that the system is an awareness and screening tool and does not constitute a clinical diagnosis or replace professional psychiatric evaluation.

### 3.9 Chapter Summary

This chapter presented the complete design, training, and integration of the AI module within the SafeSpace system. Two complementary machine learning models were developed and integrated:

The Survey DNN, a four-layer fully connected neural network trained on 39,775 DASS-42 questionnaire responses, achieves a mean absolute prediction error of less than 0.5 raw score points per subscale and a severity-level classification accuracy exceeding 88%, demonstrating that structured psychometric data can be effectively modelled by relatively compact neural architectures when appropriate preprocessing, regularisation, and target normalisation are applied.

The NLP Model, a fine-tuned XLM-RoBERTa transformer trained on the bilingual MentalHealth-Darija corpus, achieves a macro F1 score of approximately 0.85 and an overall accuracy of approximately 86% on the three-class Arabic-English mental health classification task, validating the efficacy of cross-lingual transfer learning for Arabic dialectal mental health NLP in the absence of large labelled standard-Arabic clinical corpora.

The two models are combined via a weighted linear late-fusion strategy ( $w_{\text{survey}} = 0.60$ ,  $w_{\text{text}} = 0.40$ ) that leverages the complementary signals of structured psychometric assessment and free-text affective expression. The fused output feeds a deterministic bilingual recommendation engine that translates probability scores into clinically grounded, contextually personalised, and safety-aware guidance, including a crisis detection pathway for suicidal ideation.

Together, these components constitute a technically rigorous, clinically informed, and practically deployable AI module capable of providing scalable, accessible, and linguistically inclusive mental health screening for Arabic-speaking and bilingual user populations.

---

# **CHAPTER 4**

## **SYSTEM ANALYSIS**

---

## Chapter 4: System Analysis

### 4.1 Development Methodology

SafeSpace was developed using an iterative Agile methodology organized into two-week sprints. Each sprint followed a cycle of planning, development, testing, and review. This approach allowed the team to deliver working increments regularly, incorporate feedback from supervisors and test users, and adapt to changing requirements as the project evolved.

Team coordination was facilitated through WhatsApp, which served as the primary communication channel for task allocation, progress tracking, and development discussions. Regular updates among team members ensured the timely completion of project milestones. Version control was managed using Git, with a remote repository hosted on GitHub.

### 4.2 Requirements Analysis

System requirements were determined by analyzing user workflows and clinical diagnostics. These requirements were categorized into functional behaviors (what the system must do) and non-functional specifications (how the system must perform).

### 4.3 Functional Requirements

**FR-1: User Authentication and Account Management.** The system must support user registration with email, username, and password. It must authenticate users via the Supabase authentication service and persist session tokens locally using SharedPreferences. Sensitive account information such as email addresses must not be displayed on the main profile screen; instead, such details must be accessible only through a dedicated settings sub-menu.

**FR-2: DASS-42 Clinical Assessment.** The application must present all 42 DASS-42 questions in a paginated, one-question-per-page format. Each question must offer four response options scored from 0 to 3. The system must calculate separate subscale scores for depression, anxiety, and stress using the standard item-to-subscale mappings and classify severity according to established clinical thresholds.

**FR-3: Free-Text Journaling with NLP Analysis.** The journal editor must provide separate input fields for the entry title and body content. Entries must be stored in the Supabase database with the format "Title\n\nBody". Users must be able to view, edit, and delete their journal entries. The system must support sending journal text to the backend NLP engine for emotional classification.

**FR-4: Multi-Modal Score Fusion.** The backend must accept both the 42-element survey answer vector and the journal text as inputs. It must compute separate scores using the MLP model (for survey data) and the XLM-RoBERTa model (for text data), then calculate a weighted fusion using 60 percent survey weight and 40 percent text weight.



**FR-5: Root-Cause Stressor Identification.** The backend must parse journal text against a bilingual keyword lexicon to identify the primary stressor category (academic, workplace, interpersonal, or general). The identified category must be used to select relevant coping recommendations.

**FR-6: Crisis Safety Net.** The backend must scan all incoming text for crisis keywords in English and Arabic. If a crisis keyword is detected, the system must bypass the scoring pipeline and return a crisis status that triggers an emergency overlay on the mobile client displaying helpline contact information.

**FR-7: Daily Mood Check-In.** The application must provide a daily check-in questionnaire that captures stress level, energy level, and sleep quality. Check-in data must be stored both locally and remotely and used to populate mood trend visualizations.

**FR-8: Therapeutic Coping Activities.** The application must offer interactive exercises, including box breathing with animated visual guides, a 5-4-3-2-1 sensory grounding game, guided meditation, bubble pop and color match stress-relief mini-games, and morning and evening routine recommendations.

**FR-9: Psychoeducational Content.** The Explore tab must provide educational articles on anxiety, depression, and stress-versus-burnout, with full-text reading views. Daily wellness challenges, such as hydration goals and gratitude journaling, must also be available.

## 4.4 Non-Functional Requirements

**NFR-1: Performance.** API response times for the combined analysis endpoint must not exceed 20-30 seconds under standard mobile network conditions. Interactive animations must render at 60 frames per second on mid-range devices.

**NFR-2: Security and Privacy.** All network communication must use HTTPS. User passwords must be hashed. Journal entries must be accessible only by their author through database-level access controls. Email addresses must be hidden from the profile screen.

**NFR-3: Reliability.** The backend service must maintain high availability through Hugging Face Spaces container management. The mobile application must cache essential data locally to remain partially functional during network outages.

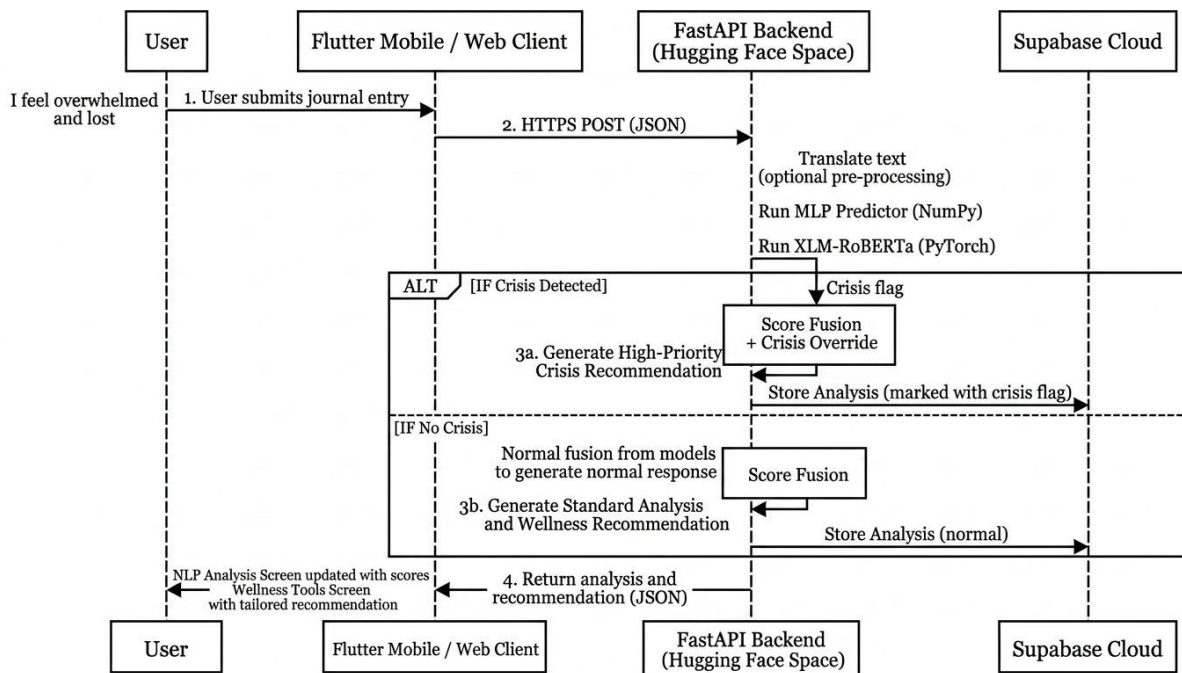
**NFR-4: Cross-Platform Compatibility.** The application must compile and run correctly on both Android and iOS devices with a consistent layout and behavior.

## 4.5 System Workflow

Figure 4.1 presents the end-to-end sequence of interactions between the user, the Flutter mobile client, the Supabase database, and the FastAPI backend during a

## Analysis

combined assessment. The user first completes the DASS-42 questionnaire and writes a free-text journal entry. The client stores the journal entry in Supabase and forwards both the 42-element survey vector and the journal text to the backend. The backend first screens the text for crisis keywords; if a crisis is detected, it returns a crisis status that causes the client to display emergency helplines immediately, bypassing all further processing. Otherwise, the backend proceeds with the standard analysis path: the MLP predicts DASS subscale scores from the survey vector, the journal text is translated if it is written in an Arabic dialect, the XLM-RoBERTa model classifies the translated text, the two sets of scores are fused using the 60/40 weighting described in Section 3.6, root stressors are extracted from the text, and the resulting scores and recommendations are returned to the client for display on the results dashboard.



[Figure 4.1 — System workflow sequence diagram, illustrating the crisis-detection branch and the normal MLP / translation / XLM-RoBERTa / fusion/recommendation processing path]

---

# **CHAPTER 5**

## **SYSTEM DESIGN**

---

## Chapter 5: System Design

### 5.1 Introduction

The visual identity of SafeSpace was designed to reflect the application's therapeutic purpose. The interface uses a dark color scheme built around deep indigo (#0D0720) and muted violet tones (#1A1035, #221545) to create a visually restful environment. Primary interactive elements use a rich purple (#7B3FE4) with lighter accent variants (#9B6FFF, #B99EFF) to draw attention without visual harshness. Feedback colors follow clinical conventions: green for normal states, yellow for mild concern, orange for moderate concern, and red for severe or critical states.

Typography uses the SF Pro Display font family for clean readability at all sizes. Cards and containers employ rounded corners (12–20 pixel radii) and subtle border highlights to establish visual hierarchy without sharp edges. Transitions between screens use smooth ease-in-out curves to reduce cognitive friction. The overall design principle is that a user who is already feeling anxious or overwhelmed should find the interface calming rather than stimulating.

### 5.2 Application Navigation Structure

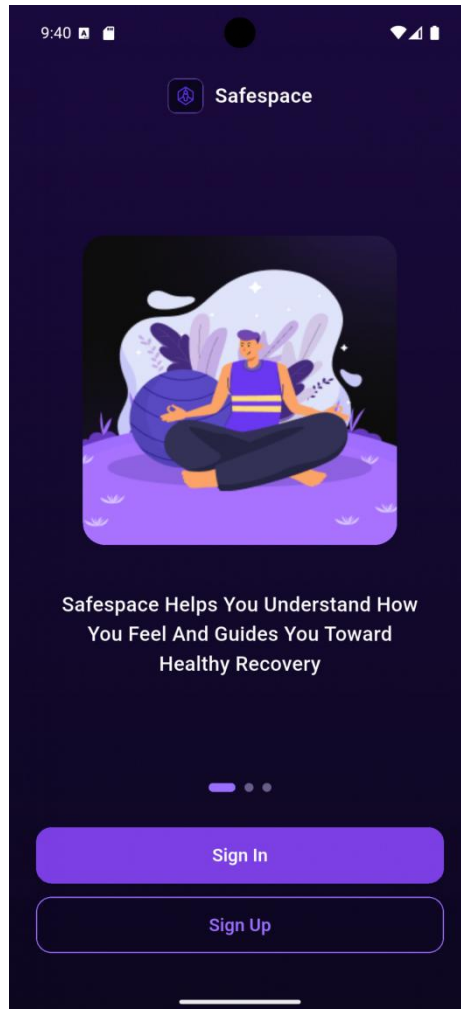
The application uses a bottom navigation bar with four primary tabs:

1. **Home Tab:** The main dashboard displays a daily motivational quote, action buttons for check-in, assessment, and NLP analysis, a daily plan section with links to Morning Ritual, Safe Journal, ADHD Exercise, and Nightly Unwind, and a personal goal tracker.
2. **Explore Tab:** Contains psychoeducational content cards on anxiety, depression, and stress-versus-burnout, each linking to a full-text article reader. Also includes daily wellness challenges such as hydration goals and gratitude journaling.
3. **Wellness Tab:** Provides access to therapeutic tools (box breathing, 5-4-3-2-1 grounding, meditation) and stress-relief mini-games (Bubble Pop, Color Match).
4. **Profile Tab:** Displays the user's name and avatar, with navigation links to Settings and Help Center, and a logout button. Sensitive account details, such as email, are intentionally excluded from this screen and accessible only through the Settings sub-menu.

## 5.3 Screen-by-Screen Design

### 5.3.1 Splash and Onboarding Screens

The splash screen displays the SafeSpace logo while the application initializes



*Figure 5.1.* Splash and Onboarding Screen

Supabase and loads cached user data from SharedPreferences. If the user has not completed onboarding, they are routed to a three-page onboarding slider. Each page displays a custom illustration paired with a caption describing a core feature: emotional understanding, healthy habit building, and mood tracking. Users can swipe through pages or skip directly to registration.

### 5.3.2 Registration and Authentication Screens

The sign-up screen collects username, email, and password. Input validation enforces non-empty fields and email format checks. Upon successful registration via the backend API, the user's credentials are cached locally, and they are routed to the home screen. The sign-in screen provides returning users with email and password fields. Both screens use the application's dark card styling with purple accent buttons.

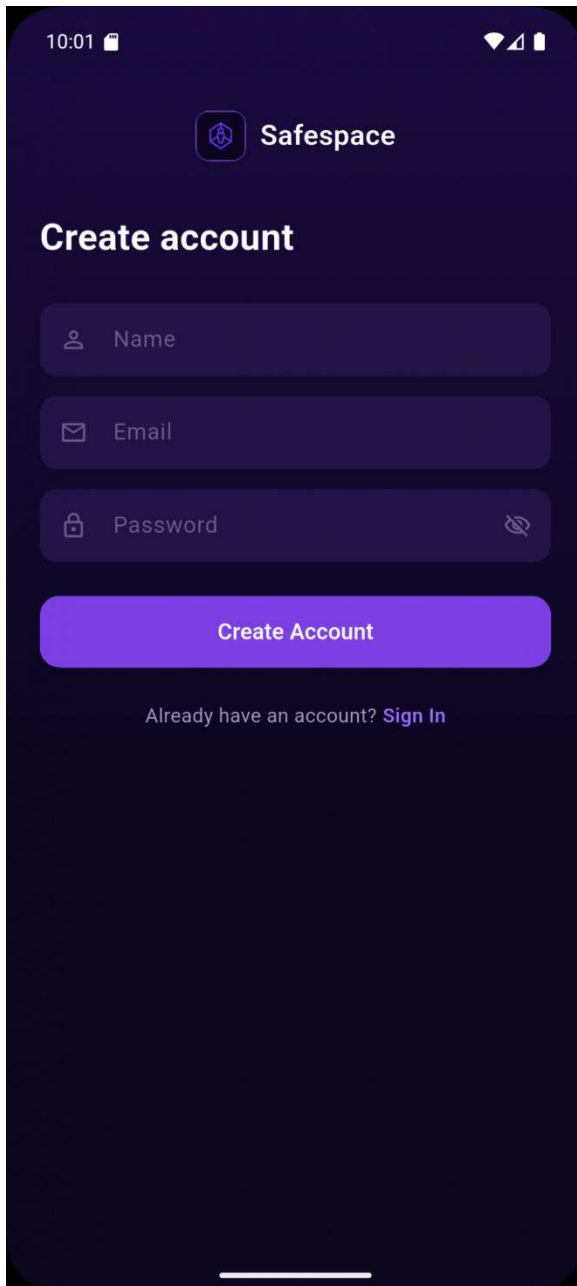


Figure 5.2. Sign-up screen

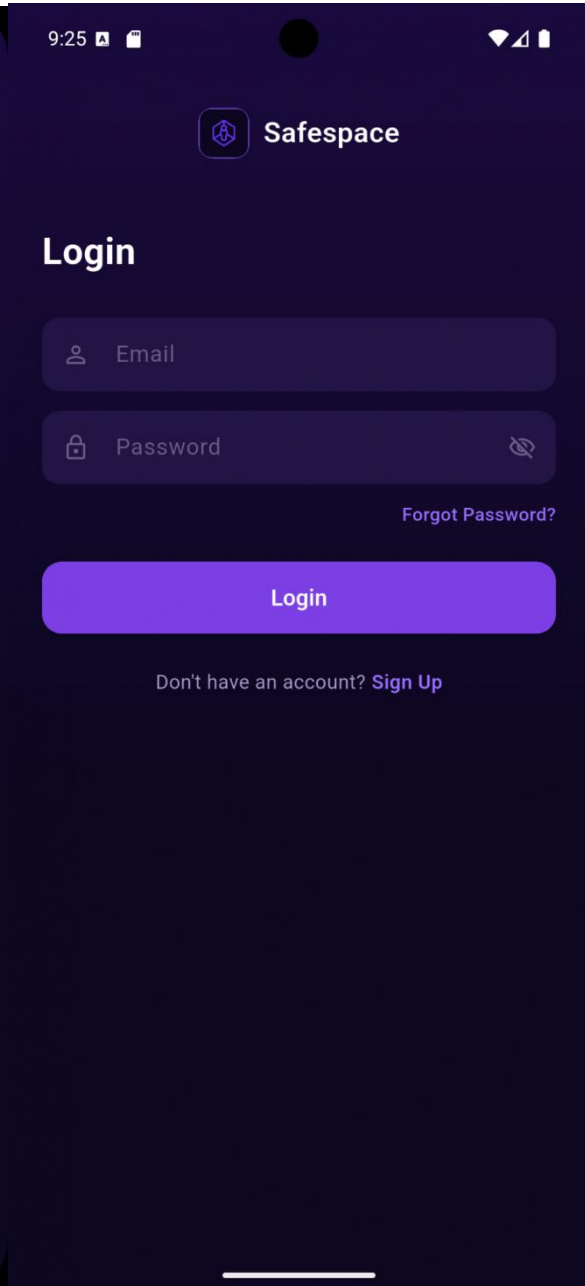


Figure 5.3. Sign-in screen

### 5.3.3 Home Dashboard

The home screen greets the user by name and displays the current date. A motivational quote card is presented below the header. Three action buttons provide quick access to the daily mood check-in, the full DASS-42 assessment, and the standalone NLP text analysis screen. Below these buttons, a daily plan section lists four activity cards: Morning Ritual, Safe Journal, ADHD Exercise, and Nightly Unwind. Each card navigates to a dedicated screen. A goal tracker section at the bottom allows users to add, complete, and remove personal goals.

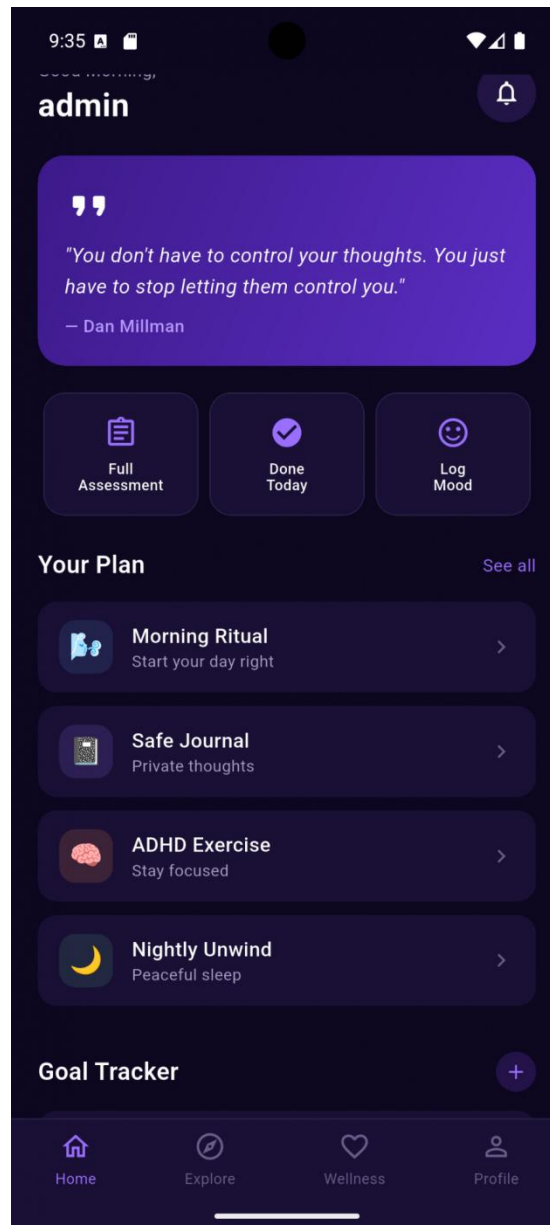


Figure 5.4. Home screen

### 5.3.4 Daily Check-In Screen

This screen captures three daily metrics through dedicated question cards. The first question measures stress level on a five-point scale using emoji-labeled options ranging from "Very stressed" to "Calm." The second question measures energy level using a continuous slider from 0 to 100 percent. The third question measures sleep duration using predefined hour ranges (less than 4 hours through 8 or more hours). Upon submission, the data is saved locally via `SharedPreferences` and synced to the Supabase database through the check-in API endpoint.

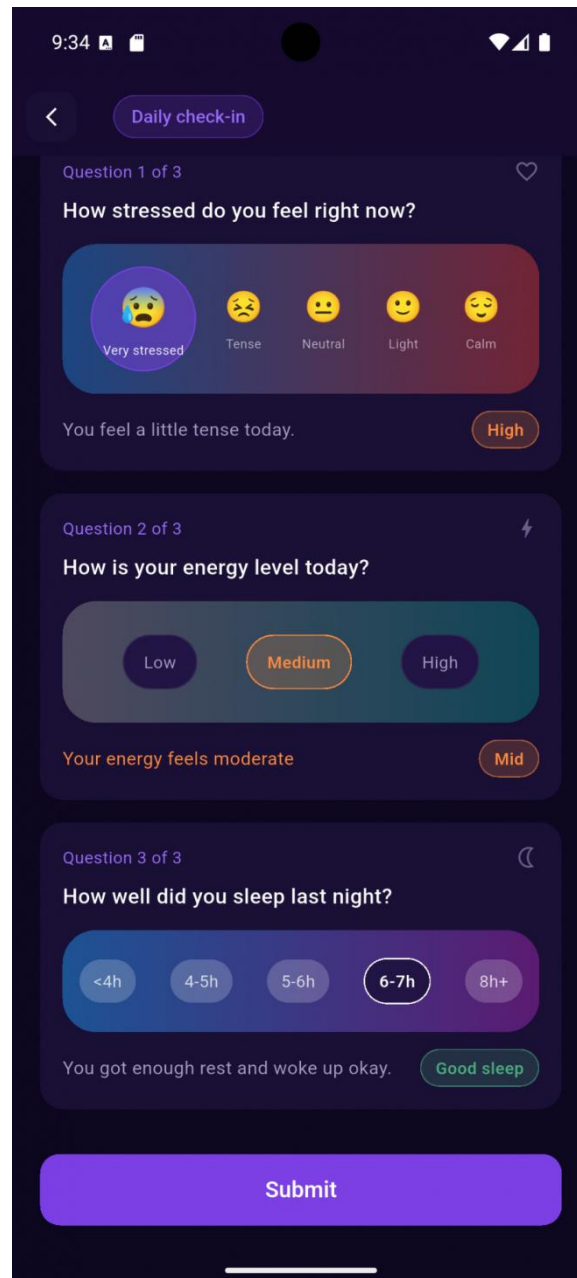


Figure 5.5. Daily Check-In Screen



### 5.3.5 DASS-42 Assessment Screen

The assessment begins with an introduction screen that explains the purpose and format of the DASS-42 instrument. When the user proceeds, the 42 questions are presented one per page using a PageView widget. Each question displays the item text and four response cards labeled "Did not apply to me at all" (0) through "Applied to me very much" (3). Selecting an option automatically advances to the next question after a 300-millisecond delay. After all 42 questions, a text input field allows the user to write a free-form description of their feelings. This text is sent alongside the survey vector to the backend analysis endpoint.

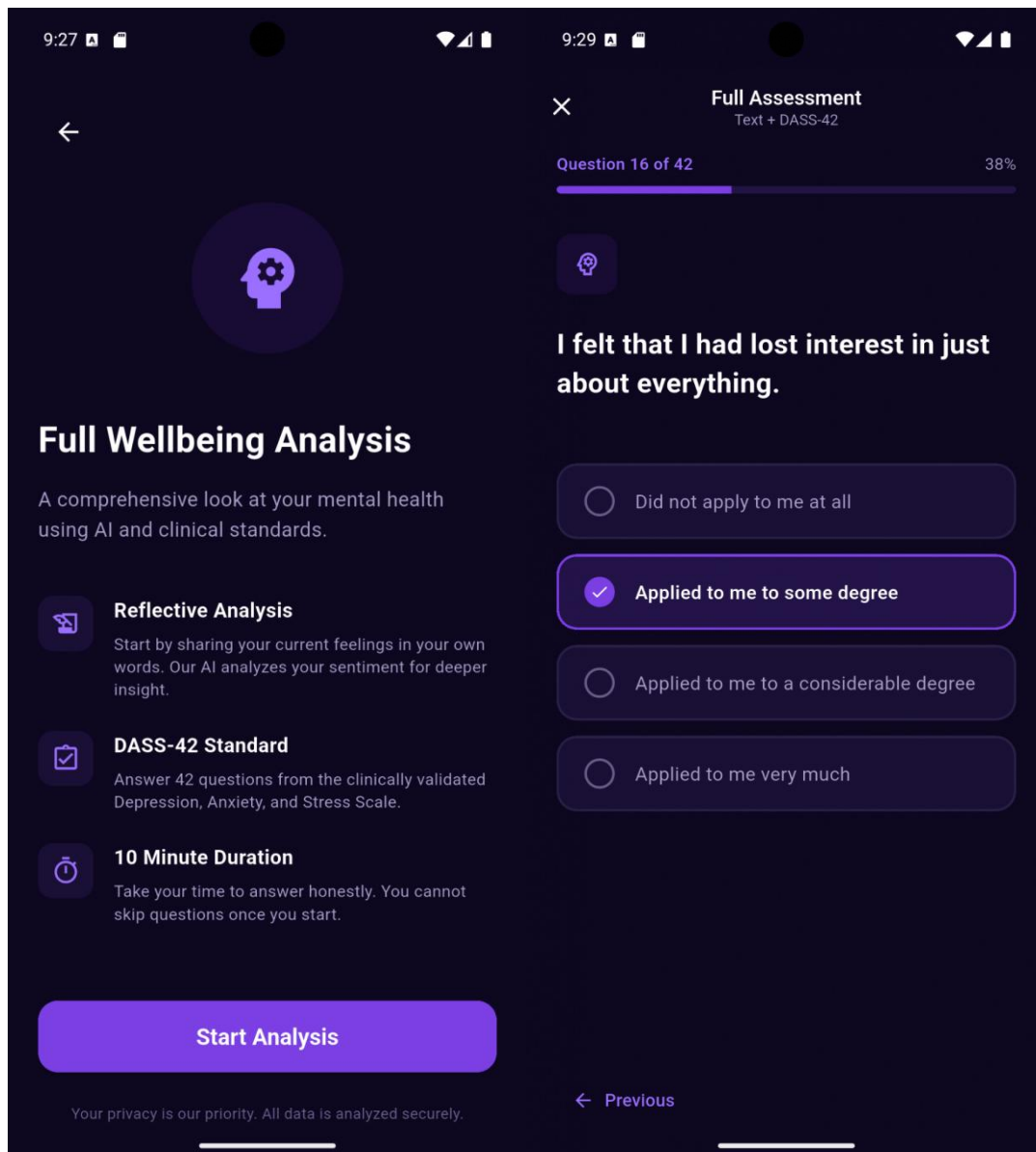


Figure 5.6. assessment boarding Screen

Figure 5.7. survey q16 Screen

### 5.3.6 DASS Results Screen

Upon receiving the backend response, this screen presents the assessment results. Three horizontal progress bars display the raw subscale scores for depression, anxiety, and stress, each color-coded by severity level. If the backend API returned successfully, additional information is displayed: the primary condition identified, the severity classification, the detected root cause of distress, tailored tips, resource links, and referral recommendations. A suicidal flag indicator is also checked; if active, a prominent crisis intervention notice is displayed.

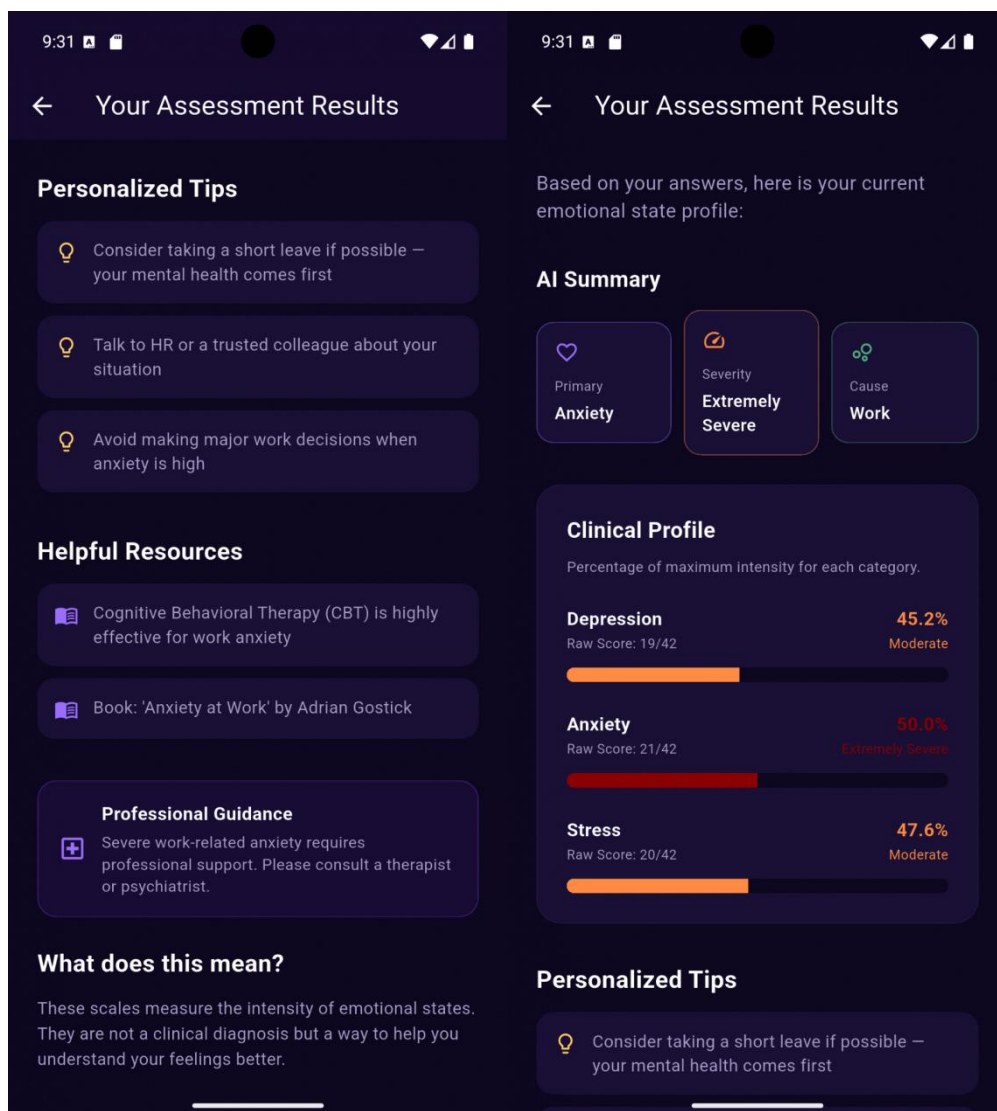


Figure 5.8. assessment Results tips

Figure 5.9. assessment Results overview

5.3.7 NLP Text Analysis Screen

This standalone screen allows users to submit free-text input for sentiment analysis without completing the full DASS-42 survey. It features a multi-line text input field with a hint indicating that both Arabic and English are supported. A microphone button activates the device's speech-to-text engine, allowing users to dictate their thoughts vocally. Upon submission, the text is sent to the backend analysis endpoint with zeroed survey answers. The returned text scores are displayed as three labeled progress bars showing depression, anxiety, and stress probabilities. An informational note explains that these are raw model probabilities.

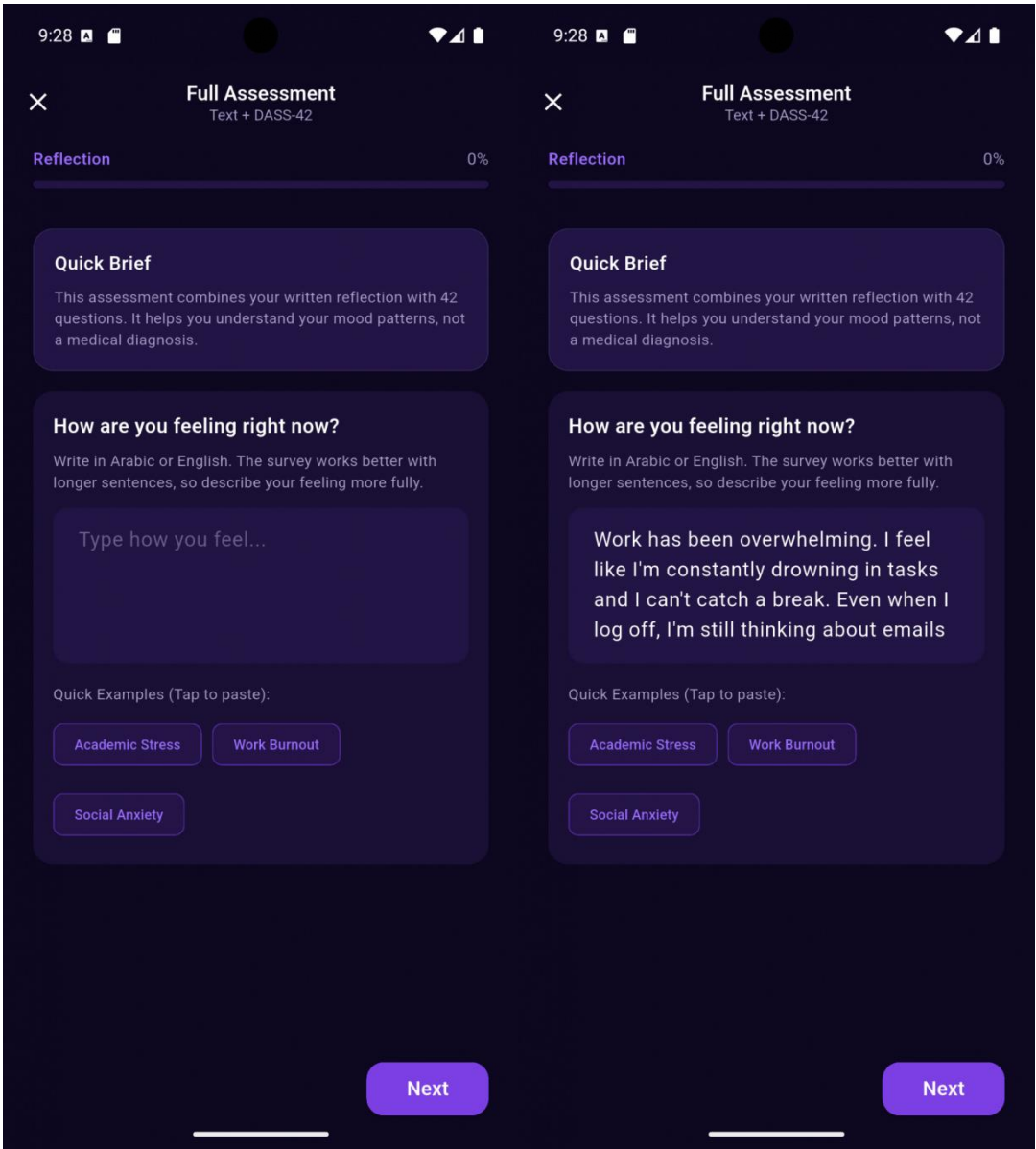


Figure 5.10. NLP Analysis Screen

Figure 5.11. NLP with Text Analysis Screen(2)

### 5.3.8 Journal Editor Screen

The journal screen provides a structured writing interface with separate text fields for the entry title and body content. A save button formats the input as "Title\n\nBody" and stores it in the Supabase database via the journal API endpoint. Below the editor, a scrollable list displays previous journal entries with the entry date, title, and a body preview that excludes the title to avoid duplication. Each entry card provides edit and delete actions. The edit dialog separates the title and body back into individual fields for convenient modification.

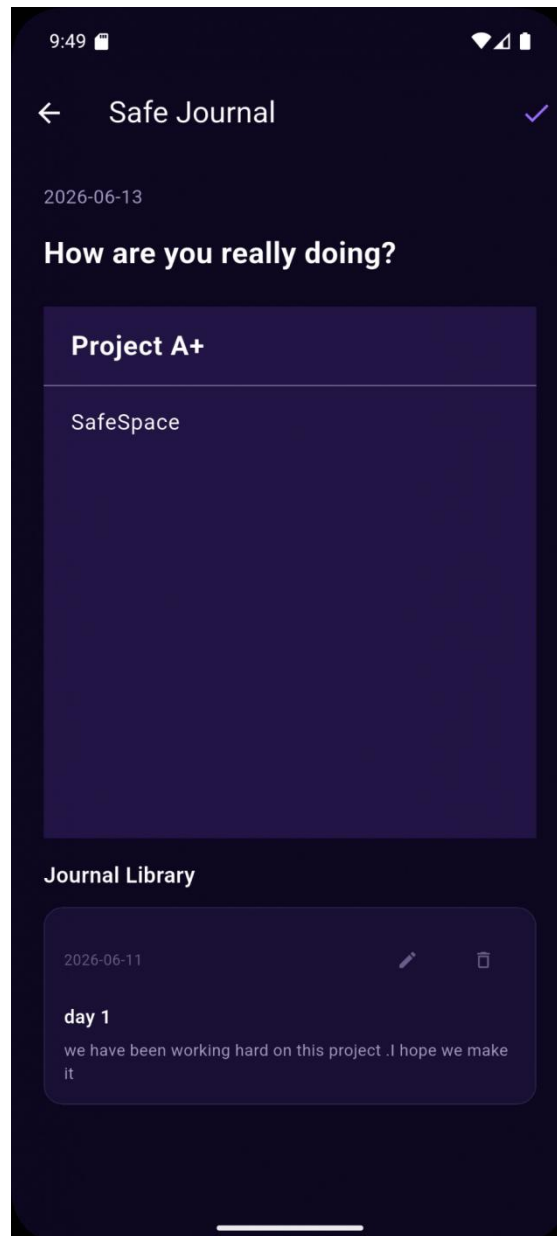


Figure 5.12. Journal Editor Screen

### 5.3.9 Wellness Tools and Mini-Games

The box breathing screen uses a pulsating animated circle that expands and contracts on a timed cycle (inhale 4 seconds, hold 4 seconds, exhale 4 seconds, hold 4 seconds), with on-screen text instructions that update with each phase. The grounding screen implements the 5-4-3-2-1 technique as a gamified selection interface: users choose from suggested sensory items (e.g., "Phone," "Desk," "Window" for the "See" step) with chip-based selection controls. The meditation screen provides a guided session with a timer and ambient instructions. Bubble Pop presents tappable floating circles that burst on contact, and Color Match challenges users to quickly identify matching colors, both designed as brief cognitive distraction exercises.

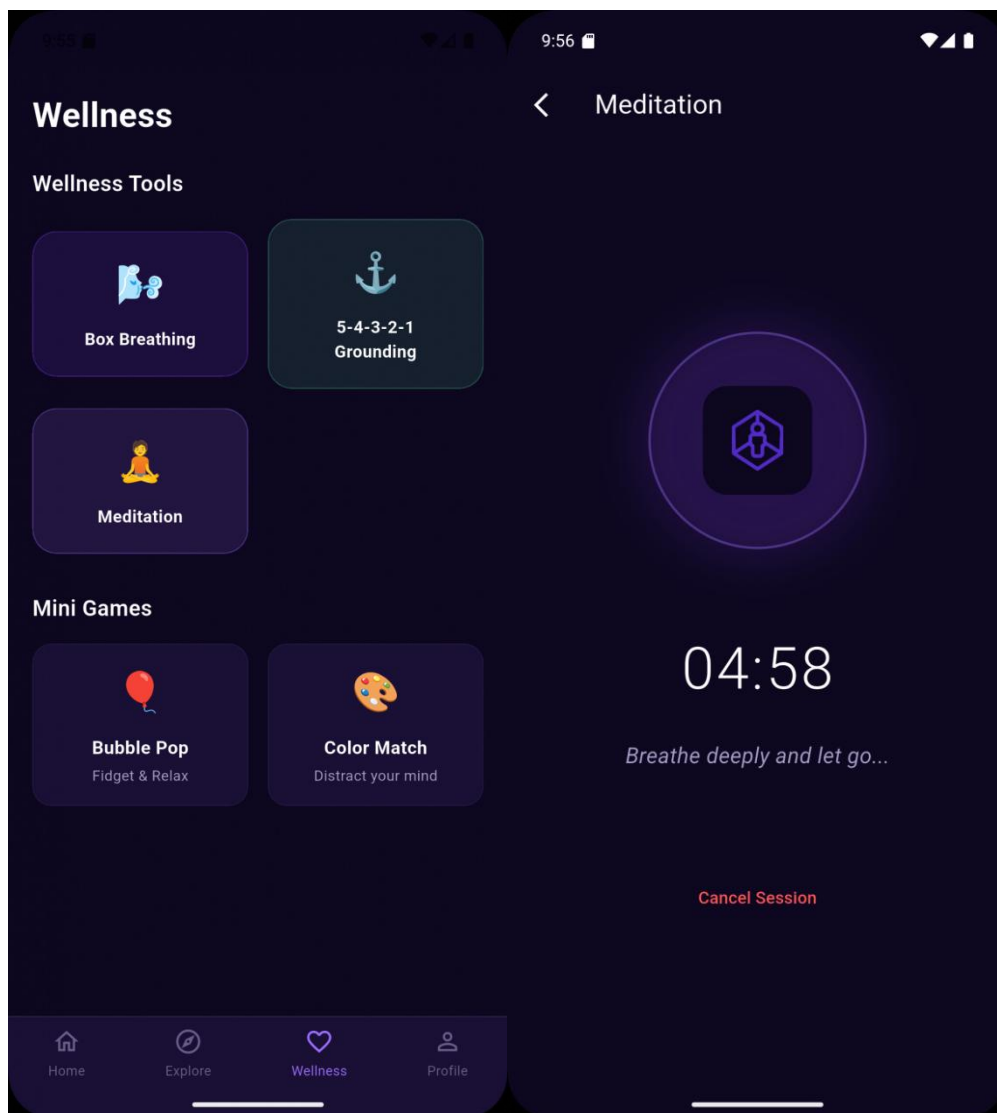


Figure 5.13. Wellness Menu Screen

Figure 5.14. meditation Screen

### 5.3.10 Morning Ritual and Nightly Unwind Screens

These recommendation screens dynamically generate activity suggestions based on the user's most recent DASS-42 results. If the user has elevated stress scores, morning recommendations include calm breathing and morning journaling. If anxiety scores are elevated, suggestions include daily intention setting and mindful beverage consumption. If depression scores are elevated, physical movement and music-based mood lifting are suggested. Evening recommendations similarly adapt, suggesting digital detox, progressive muscle relaxation, grounding exercises, positive reflection, and self-compassion affirmations. If no DASS results are available, a set of general wellness recommendations is displayed.

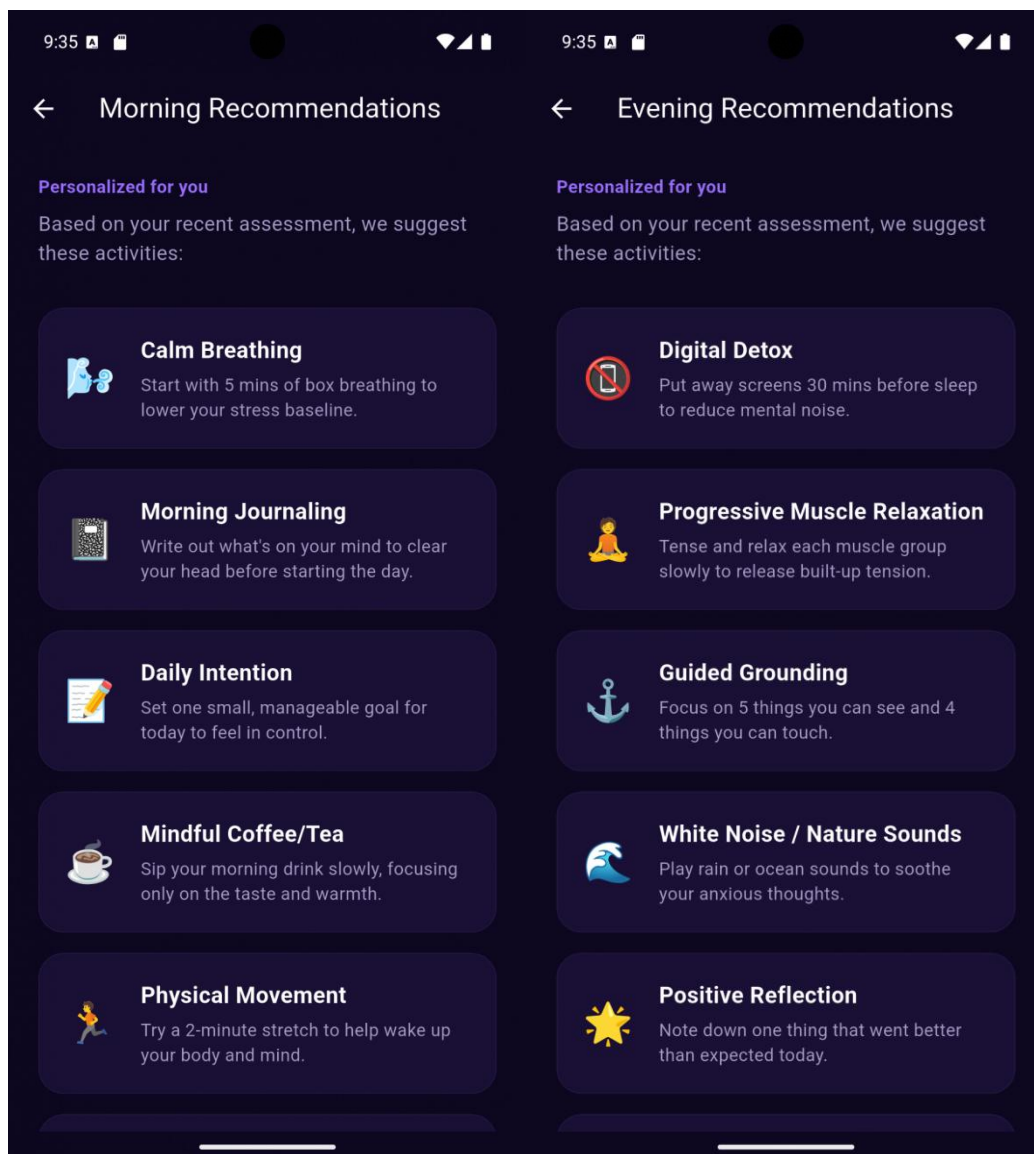


Figure 5.15. Morning Ritual Screen

Figure 5.16. Nightly Unwind Screen

### 5.3.11 Mood Patterns and Trend Visualization Screen

The mood patterns screen provides longitudinal visual analytics of the user's emotional trajectory. It fetches both local check-in history from SharedPreferences and remote check-in records from the backend API, merging them into a unified dataset. Three interactive line charts, rendered using the `fl_chart` package, display stress level, energy level, and sleep quality over configurable time windows of 7, 14, or 30 days. Each chart uses color-coded data points and smooth Bézier curves to highlight trends. A separate section fetches the user's DASS-42 analysis history from the `/analyses/history` endpoint and displays historical depression, anxiety, and stress scores as overlaid trend lines, enabling users to observe how their clinical indicators evolve across multiple assessments.

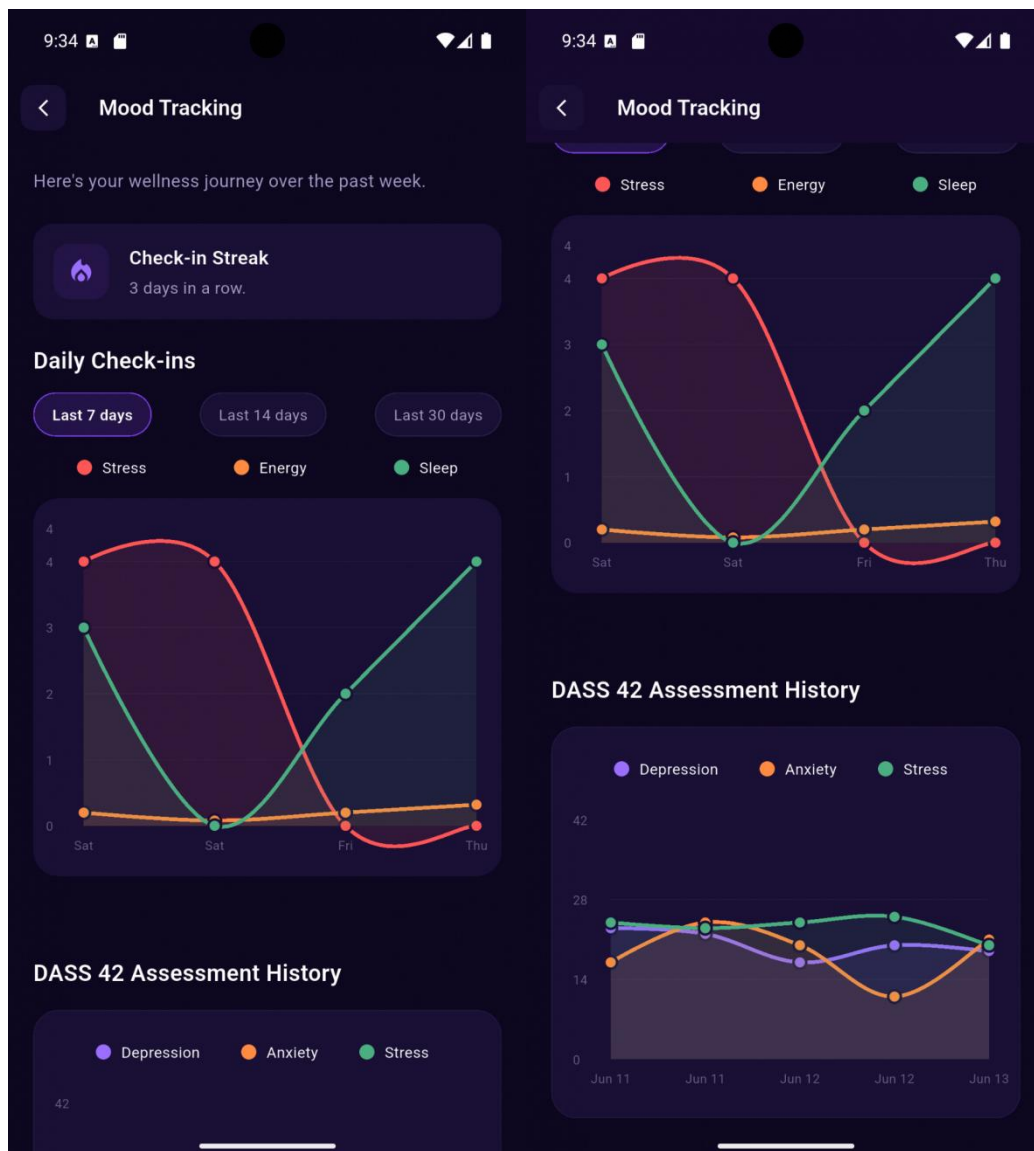


Figure 5.17. Mood Patterns Screen(1)

Figure 5.18. Mood Patterns Screen(2)



### 5.3.12 Settings Screen

The settings screen is organized into three sections: Account Information (displaying username and email), Security (providing a password reset option that sends a reset link via Supabase), and Preferences (notification settings and language selection). This screen is the only location where the user's email address is visible, fulfilling the privacy requirement that sensitive credentials not appear on the main profile view.

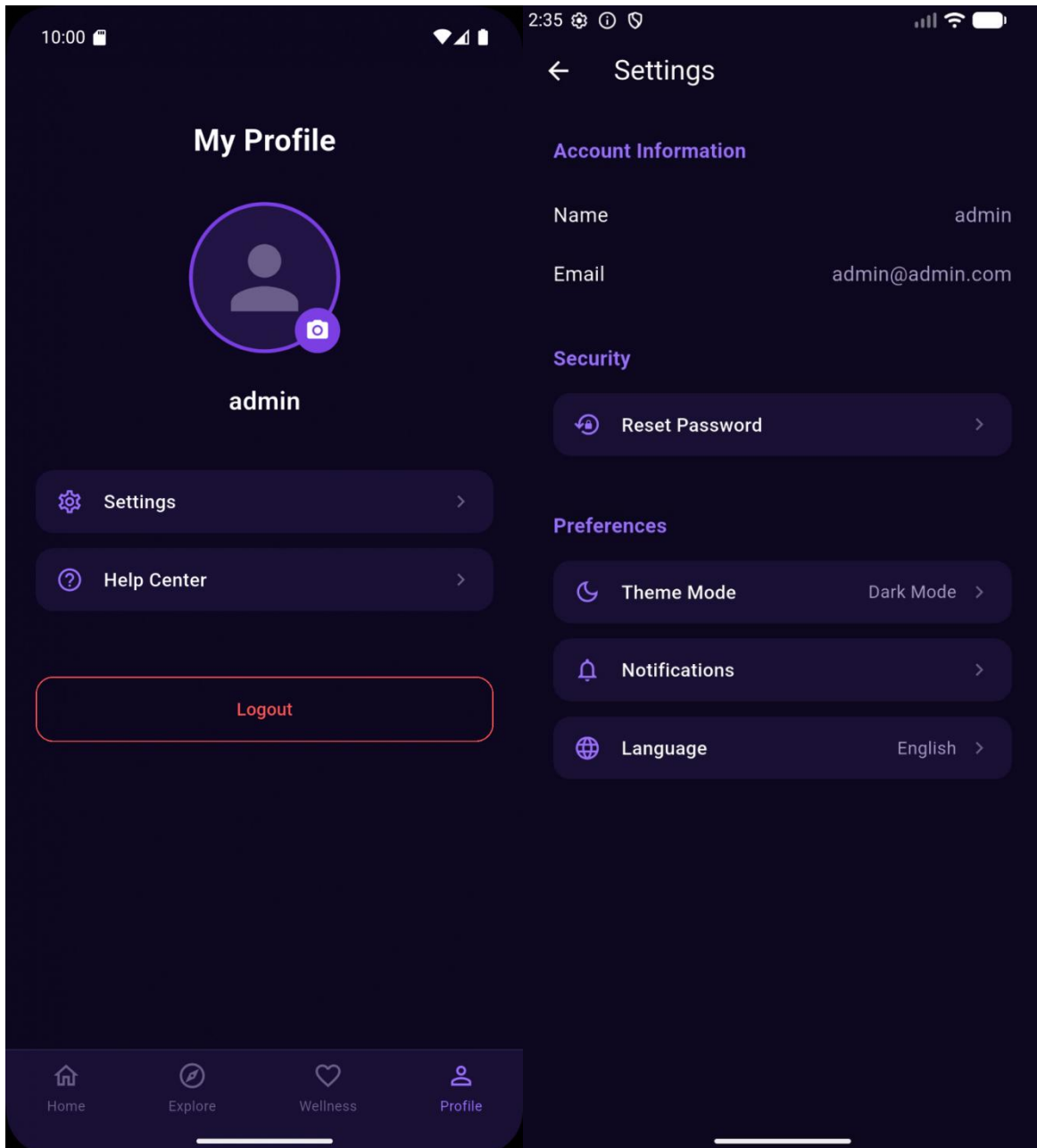


Figure 5.19. Profile Screen

Figure 5.20. Settings Screen



## 5.4 Database Schema Design

The PostgreSQL database managed through the FastAPI backend contains four primary tables: `users`, `journal_entries`, `checkins`, and `analyses`. Table 5.1 summarises the schema of each.

| Table                        | Key Columns                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>users</code>           | <code>id</code> (PK, auto-increment); <code>name</code> (nullable); <code>email</code> (unique, indexed); <code>password</code> (hashed); <code>created_at</code>                                                                                                                                                                                                                                                                                                                                        |
| <code>journal_entries</code> | <code>id</code> (PK, auto-increment); <code>user_id</code> (FK → <code>users.id</code> ); <code>content</code> (TEXT, format "Title\n\nBody"); <code>created_at</code> ; <code>updated_at</code> (nullable)                                                                                                                                                                                                                                                                                              |
| <code>checkins</code>        | <code>id</code> (PK, auto-increment); <code>user_id</code> (FK, indexed); <code>mood</code> (INT); <code>sleep</code> (INT); <code>energy</code> (FLOAT); <code>created_at</code>                                                                                                                                                                                                                                                                                                                        |
| <code>analyses</code>        | <code>id</code> (PK, auto-increment); <code>user_id</code> (FK, indexed); <code>primary_condition</code> ; <code>clinical_scoring</code> (JSON); <code>text_input</code> ; <code>text_input_hash</code> ; <code>text_scores</code> (JSON); <code>survey_scores</code> (JSON); <code>fused_scores</code> (JSON); <code>severity</code> ; <code>cause</code> ; <code>suicidal_flag</code> (BOOLEAN); <code>model_version</code> ; <code>app_version</code> ; <code>locale</code> ; <code>created_at</code> |

Table 5.1 — SafeSpace PostgreSQL schema overview

The `analyses` table is the richest entity in the schema. It stores the complete audit trail of every assessment: the raw text input (with a SHA-256 hash for deduplication), the individual model outputs (`text_scores` and `survey_scores` as JSON), the fused result, the detected severity level, root cause category, and suicidal flag. The `clinical_scoring` column stores the DASS-42 subscale breakdown as a JSON object with nested score and severity fields for each of depression, anxiety, and stress. Composite indexes on (`user_id`, `created_at`) are defined on both the `analyses` and `checkins` tables to optimize time-series queries for the mood patterns screen.

The `journal_entries.content` field stores the combined title and body text in the format "Title\n\nBody". This concatenated format was chosen to maintain backward compatibility with the backend analysis endpoint, which expects a single text string. The mobile client parses this format on retrieval, extracting the first line as the title and the remaining lines as the body content. The `updated_at` column is populated whenever a journal entry is modified through the PUT endpoint.

Personal goals (created through the Home dashboard goal tracker) are stored locally in `SharedPreferences` on the device rather than in PostgreSQL, and are scoped to the current `user_id` via dynamically constructed cache keys (e.g., `goals_${userId}`), ensuring that switching accounts does not expose another user's data. The full implementation of the data-access layer that reads and writes these tables — including the SQLAlchemy ORM models, connection pooling configuration, and table-creation timeout handling — is presented in Section 6.4.

## 5.5 System Architecture Overview

Having established the screen-level design (Section 5.3) and the data model (Section 5.4), this section presents the high-level architectural view that ties the two together. SafeSpace follows a modular four-layer architecture that separates presentation, business logic, data management, and artificial intelligence concerns. This section introduces each layer at a conceptual level; Chapter 6 then describes, layer by layer, the concrete technologies, code modules, and API contracts that realise this architecture.

1. **Frontend Layer** — implemented in Flutter, enabling cross-platform deployment across Android, iOS, and Web from a single codebase (detailed in Section 6.1).
2. **Backend Layer** — a FastAPI service responsible for authentication, assessment processing, recommendation generation, and orchestration of the AI models, exposed via RESTful endpoints (detailed in Section 6.2).
3. **Database Layer** — a PostgreSQL database (schema presented in Section 5.4) accessed through SQLAlchemy, plus Supabase-managed authentication (detailed in Section 6.4).
4. **Artificial Intelligence Layer** — the survey DNN and fine-tuned XLM-RoBERTa text classifier described in Chapter 3, combined via the score-fusion and crisis-override logic (implementation detailed in Section 6.3).

## Design

Figure 5.1 illustrates how these layers interconnect at runtime: the Flutter client communicates with the FastAPI backend over HTTPS/JSON for assessment, journal, and check-in operations, while communicating directly with Supabase via its SDK for authentication and session management. The backend, in turn, hosts the MLP predictor, the XLM-RoBERTa classifier, and the score-fusion and crisis-detection logic, and persists results to the PostgreSQL database.

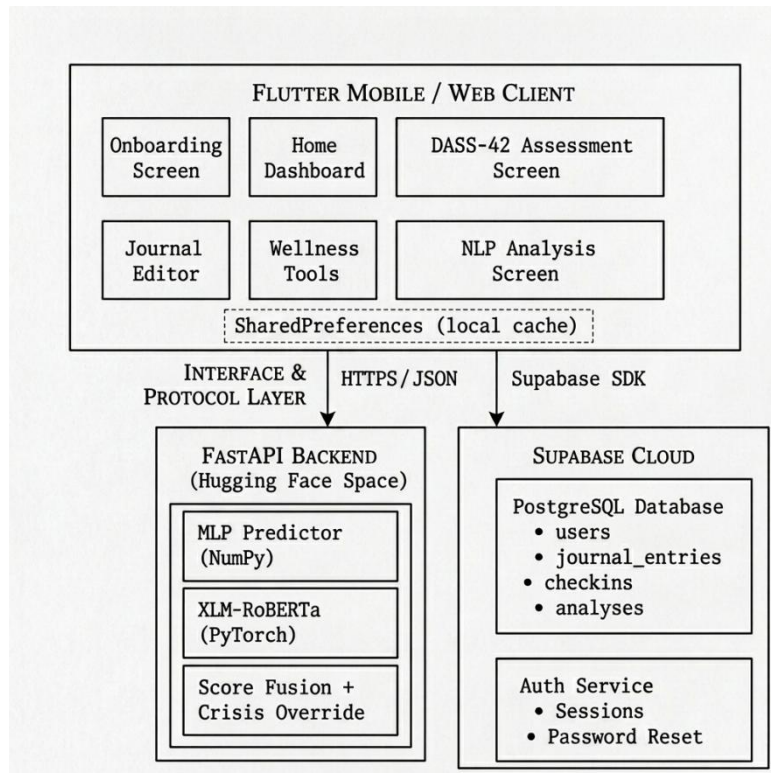


Figure 5.21 — SafeSpace high-level system architecture (four-layer overview)

Two design decisions are worth highlighting at this overview level, both elaborated further in Chapter 6. First, the AI layer is intentionally split between a lightweight NumPy-based forward pass for the survey model and a PyTorch-based transformer for the text model, allowing the survey component to run with minimal dependencies while the heavier transformer model is loaded once and cached at server startup (Section 6.2). Second, authentication is deliberately split between Supabase (session and credential management, accessed directly by the Flutter client) and the FastAPI backend (which scopes all journal, check-in, and analysis records to the authenticated user's ID), a separation of concerns that is detailed in Section 6.4.

## 5.6 Accessibility and Responsive Design

To ensure inclusivity, the application follows accessibility best practices:

1. High contrast ratios improve readability.
2. Responsive layouts support mobile, tablet, and desktop devices.
3. Large touch targets facilitate interaction.
4. Clear navigation structures reduce confusion.
5. Consistent visual hierarchy improves usability.
6. Error messages are descriptive and user-friendly.

The responsive design strategy allows the platform to adapt seamlessly across different screen sizes while maintaining a consistent user experience. The cross-platform web build of this responsive layout is described in Section 6.6.

## 5.7 Security and Privacy Considerations

Given the sensitive nature of mental health data, security and privacy are critical components of the system.

User information is protected through secure authentication mechanisms, encrypted communication channels, and controlled access policies. Assessment results remain confidential and are only accessible to authorized users.

The system follows privacy-by-design principles, ensuring that user trust remains a fundamental aspect of the platform. The specific mechanisms that implement these principles — password hashing, HTTPS enforcement, and user-scoped database queries — are described in Section 6.4.

## 5.8 Chapter Summary

This chapter presented the UI/UX design principles, screen-by-screen interface specification, database schema, and high-level architectural overview of SafeSpace. The chapter discussed visual design considerations, navigation structure, the twelve-screen user journey, the four-table PostgreSQL schema, and the four-layer architectural overview that situates the Flutter client, FastAPI backend, Supabase-managed database, and AI layer relative to one another. Chapter 6 builds directly on this overview, presenting the concrete implementation of each architectural layer.

---

# **CHAPTER 6**

## **TECHNICAL**

## **IMPLEMENTATION**

---

## Chapter 6: Technical Implementation

### 6.1 Mobile Frontend Module

**Purpose.** The mobile frontend serves as the primary interface through which users interact with the SafeSpace system. It handles all user input collection, local data caching, screen navigation, and presentation of results.

**Technologies Used.** The frontend is built with Flutter 3.x [26] using the Dart programming language. Flutter was selected because it compiles to native ARM code for both Android and iOS from a single codebase, eliminating the need to maintain separate platform-specific implementations. Additional packages include supabase\_flutter for database and authentication integration [24], http for REST API communication, shared\_preferences for local key-value caching, fl\_chart for rendering assessment result charts, speech\_to\_text for voice input on the NLP analysis screen, and intl for date formatting.

**Architecture.** The application follows a centralized state management pattern. A static AppState class in lib/data/app\_state.dart serves as the single source of truth for user identity, mood history, journal entries, goal lists, and DASS results. This class provides methods for loading data from SharedPreferences on startup, saving updated state after user actions, and syncing with the remote database through ApiService. Screens read from and write to AppState directly, ensuring consistency across the application.

**Key Implementation Details.** The DASS-42 questions are defined as a constant list of DassQuestion objects in lib/data/dass\_questions.dart, each containing an ID string and the question text. The assessment screen uses a PageController to manage the paginated question flow, a map of integer answers indexed by question number, and hardcoded arrays of item indices for each subscale (depression, anxiety, stress) to compute scores on submission, consistent with the item-to-subscale mapping specified in Section 3.2.2.

The journal screen concatenates title and body inputs as "\$title\n\n\$body" before calling AppState.addJournalEntry(). When displaying entries, AppState.refreshJournalEntries() parses the stored content by splitting on newlines: the first line becomes the title, and lines after the second become the body preview, preventing the title from appearing twice in the display.

**Security Considerations.** Session tokens managed by the Supabase SDK are stored in the device's secure storage. The profile screen intentionally omits the user's email address; this information is accessible only through the settings screen, reducing exposure of sensitive data during casual use.

### 6.2 Backend API Module

**Purpose.** The backend hosts the machine learning models and exposes REST API endpoints for authentication, analysis, check-in recording, and journal management.

**Technologies Used.** The backend is written in Python 3.9+ using FastAPI [25] as the web framework, with Uvicorn as the ASGI server. Machine learning inference uses PyTorch [22] for the XLM-RoBERTa transformer and NumPy for the MLP forward pass. The entire backend is containerized using Docker and deployed on Hugging Face Spaces [27], which provides free-tier hosting with automatic container management.

**API Endpoints.** The ApiService class on the Flutter client communicates with the following endpoints hosted at <https://alisakr9997-safespace.hf.space/api/v1> :

| Endpoint          | Method | Purpose                                             |
|-------------------|--------|-----------------------------------------------------|
| /auth/signup      | POST   | Register a new user with name, email, and password  |
| /auth/login       | POST   | Authenticate and return user credentials            |
| /analyze          | POST   | Accept survey answers and text, return fused scores |
| /chat             | POST   | Forward messages to a chatbot proxy                 |
| /checkin          | POST   | Record a daily mood check-in                        |
| /checkin/history  | GET    | Retrieve check-in history for a user                |
| /journal          | POST   | Create a new journal entry                          |
| /journal/history  | GET    | Retrieve journal entries for a user                 |
| /journal/{id}     | PUT    | Update an existing journal entry                    |
| /journal/{id}     | DELETE | Delete a journal entry                              |
| /analyses/history | GET    | Retrieve past analysis results                      |

Table 6.1 — SafeSpace backend REST API endpoints

**Performance Considerations.** The XLM-RoBERTa model is loaded into memory once at server startup and cached using Python's @lru\_cache decorator and Streamlit's @st.cache\_resource annotation. This eliminates repeated disk reads and model initialization, reducing inference latency from over 8 seconds on cold start to under 1.5 seconds for cached requests.

## 6.3 AI Model Implementations

### 6.3.1 DASS-42 Multi-Layer Perceptron

The MLP model was trained on 39,775 labeled DASS-42 survey responses. The architecture consists of an input layer of 42 nodes (one per question), a hidden layer of 64 nodes with ReLU activation, and an output layer of 3 nodes representing depression, anxiety, and stress probabilities with softmax activation.

**Answer Scale Shifting.** An important preprocessing step occurs before the survey vector reaches the MLP. The Flutter client collects answers on a 0–3 Likert scale (matching the standard DASS-42 format described in Section 3.2.1), but the MLP was trained on a dataset that uses a 1–4 scale. The backend, therefore, shifts every answer by adding 1 before inference:

This single-line transformation is critical: omitting it would cause the model to systematically underestimate severity, since all input values would be one unit lower than the training distribution.

**Feature Scaling.** After shifting, the answer vector is normalized using a pre-fitted `StandardScaler` (`scaler.pkl`) that was saved during training (Section 3.4.2). This scaler transforms the 42-dimensional input to have zero mean and unit variance, matching the distribution that the model weights expect.

**Lightweight Inference.** To minimize deployment dependencies, the trained weights and biases were exported to a serialized pickle file (`model_weights.pkl`). At inference time, the backend loads these parameters and performs raw matrix multiplication using NumPy.

This approach avoids importing PyTorch or TensorFlow for the survey model, keeping the inference footprint minimal.

### 6.3.2 XLM-RoBERTa Text Classifier

The text classification pipeline processes journal entries through five stages:

1. **Text Cleaning:** The input text is normalized by collapsing repeated characters (e.g., "soooo" → "soo"), removing punctuation and special characters while preserving Arabic Unicode characters (`\u0600–\u06FF`), and collapsing whitespace.
2. **Dialect Translation:** The cleaned text is translated to English using GoogleTranslator (via the `deep-translator` library), which wraps the Google Translate API. The translated English text and the original text are concatenated with a [SEP] separator token to form a bilingual input: "english\_translation [SEP] original\_text". This dual-input strategy ensures the model receives both the semantically standardized English version and the original dialectal nuances.
3. **Tokenization:** The combined text is tokenized using the XLM-RoBERTa tokenizer with padding, truncation at 192 tokens, and conversion to PyTorch tensors.
4. **Classification:** The tokenized input is passed through the fine-tuned XLM-RoBERTa model in inference mode (`torch.no_grad()`). The output logits are converted to probabilities via softmax, yielding raw scores for depression, anxiety, and stress.
5. **Keyword Boost Post-Processing:** The raw model probabilities are passed through a keyword boost function that corrects for an observed stress-bias in the XLM-RoBERTa output.



Keyword Boost System. During testing, it was observed that the fine-tuned XLM-RoBERTa model [14] exhibited a systematic bias toward predicting “stress” even when the input text contained explicit depression or anxiety indicators. To compensate, a post-processing keyword\_boost() function scans the original text and its English translation against three bilingual keyword lexicons:

1. **Depression keywords (56 terms):** Including Arabic dialect variants such as مكتئب (depressed), مفيش أمل (no hope), نفسيتي وحشة (my mood is terrible), and English terms such as “hopeless,” “worthless,” and “numb.”
2. **Anxiety keywords (27 terms):** Including قلقان (anxious), هلع (panic), وسواس (OCD), and English terms such as “panic,” “restless,” and “phobia.”
3. **Stress keywords (13 terms):** Including مضغوط (pressured), إجهاد (exhaustion), and English terms such as “burnout” and “overwhelmed.”

The function counts keyword hits for each category. If the dominant keyword category differs from the model's top prediction, the function overrides the probability distribution by boosting the keyword-matched condition to a minimum of 55% (increasing by 10% per additional keyword hit, capped at 85%) and proportionally redistributing the remaining probability among the other two conditions. The final scores are normalized to sum to 1.0. This mechanism ensures that when a user explicitly writes “I feel depressed and hopeless,” the system does not incorrectly classify it as stress.

### 6.3.3 Score Fusion and Crisis Override

The fusion algorithm combines both model outputs using the formula introduced in Section 3.6.2:

$$S_{fused} = 0.60 \times S_{survey} + 0.40 \times S_{text}$$

Suicidal Ideation Detection. Before any scoring is computed, the detect\_suicidal() function scans all incoming text against a lexicon of 28 bilingual crisis phrases. These include direct expressions such as انتحار (suicide), أقتل نفسي (kill myself), عايز أموت (I want to die), مش عايز أعيش (I don't want to live), إيذاء النفس (self-harm), and لو مت (if I die), as well as English equivalents including “end my life,” “hurt myself,” “want to disappear,” and “can't go on.” If any crisis keyword matches, the entire scoring pipeline is bypassed. The API returns a crisis response with:

1. *suicidal\_flag*: true
2. *severity*: "crisis"
3. *Emergency tips in both English and Arabic* (e.g., “You are not alone — help is available right now” / أنت لست وحدك — المساعدة متاحة الآن)

International crisis hotline resources, including the International Association for Suicide Prevention's Crisis Centres directory [29], Crisis Text Line (US: text HOME to 741741) [30], Befrienders Worldwide [31], Egypt's national Ministry of Health and Population mental-health support hotline (08008880700), and Saudi Arabia's Musanadah line (920033360)

An urgent referral message directing the user to contact a crisis line or visit the nearest emergency room is also included. On the mobile client, this crisis response triggers a prominent modal overlay that supersedes the normal results display.

**Root-Cause Stressor Extraction.** If no crisis is detected, the fused scores are computed, and the `extract_cause()` function parses the text against a bilingual keyword lexicon organized into nine cause categories, summarised in Table 6.2.

| Category      | Example Keywords (EN)                  | Example Keywords (AR)        |
|---------------|----------------------------------------|------------------------------|
| Work          | boss, deadline, salary, overtime       | شغل, مدير, راتب, وظيفة       |
| Relationships | breakup, divorce, loneliness, conflict | حبيب, طلاق, وحيد, خلاف       |
| Financial     | debt, broke, rent, unemployment        | فلوس, دين, إيجار, بطالة      |
| Academic      | exam, grades, university, study        | امتحان, درجات, جامعة, مذاكرة |
| Health        | pain, hospital, medication, sleep      | وجع, مستشفى, دواء, نوم       |
| Social        | judgment, shy, isolated, can't talk    | حكم, خجل, منعزل              |
| Self-Worth    | failure, worthless, not enough, loser  | فاشل, لا قيمة, مش كافي       |
| Trauma        | abuse, nightmares, death, accident     | صدمة, إساءة, كوابيس, وفاة    |
| General       | (fallback when no keywords match)      | (fallback)                   |

Table 6.2 — Bilingual cause-extraction keyword lexicon

The function counts keyword hits per category and selects the category with the highest count. If no keywords match, the cause defaults to “general.” The detected cause is passed to the recommendation engine (Section 6.5) to select domain-specific coping advice.

## 6.4 Database and Authentication Module

**Purpose.** Supabase [24] provides the authentication service for the Flutter client, while the FastAPI backend manages its own PostgreSQL connection for data persistence through SQLAlchemy ORM.

**Technologies Used.** The Flutter client connects to Supabase via the `supabase_flutter` SDK for authentication (signup, login, password reset, session management). The FastAPI backend connects directly to the PostgreSQL database using SQLAlchemy with connection pooling (`pool_pre_ping=True`) and a 5-second connection timeout.

Database tables — corresponding to the schema presented in Table 5.1 — are created automatically at server startup via `Base.metadata.create_all()` wrapped in an 8-second async timeout to prevent blocking the server if the database is temporarily unreachable.

**Security Considerations.** User passwords are hashed using SHA-256 before storage. The login endpoint supports both hashed password comparison and a legacy plain-text fallback for users who registered before hashing was implemented. Password reset for the mobile client is implemented through Supabase's `resetPasswordForEmail` method. All API communication uses HTTPS, and CORS middleware is configured to accept requests from any origin to support the cross-platform Flutter client.

**Integration.** The `ApiService` class on the Flutter client appends the current `AppState.userId` to all API requests, ensuring that backend operations are scoped to the authenticated user. Journal entries, check-in records, and analysis histories are all filtered by `user_id` in both API queries and database access patterns.

## 6.5 Bilingual Recommendation Engine

**Purpose.** The recommendation engine translates raw assessment outputs into actionable, personalized coping advice. It is the module that bridges the gap between the clinical scoring described in Section 3.7 and practical user guidance presented in the DASS Results screen (Section 5.3.6).

**Architecture.** The engine is implemented in `recommendations.py` (564 lines) and is structured as a hierarchical decision tree with three levels:

1. **Severity Classification:** The `get_severity()` function converts the fused model probability (0.0–1.0) into a DASS-42 raw score approximation by multiplying against the maximum possible raw score for each subscale (depression: 84, anxiety: 72, stress: 84). The approximated score is then classified against the standard DASS-42 clinical cutoffs (Table 3.1) into one of five levels: normal, mild, moderate, severe, or extremely severe. For recommendation selection, these five levels are grouped into two severity bands: `mild_moderate` (normal through moderate) and `severe` (severe and extremely severe).
2. **Root-Cause Routing:** The detected cause category (from the stressor extraction module, Section 6.3.3) is used to select a cause-specific recommendation set within the identified condition. For example, a user classified as “depression + work” receives different advice than a user classified as “depression + self-worth.” If the detected cause does not have a dedicated recommendation set, the engine falls back to the “general” recommendations for that condition.
3. **Bilingual Content Delivery:** Every recommendation entry contains four parallel content arrays: `tips_en` (practical coping tips in English), `tips_ar` (the same tips in Arabic), `resources_en` (books, apps, and therapeutic techniques in English), `resources_ar` (the same resources in Arabic), plus bilingual referral messages indicating when to seek professional help.

Recommendation Database Coverage:

| Condition  | Cause Categories Covered     |
|------------|------------------------------|
| Anxiety    | Work, Relationships, General |
| Depression | Work, Self-Worth, General    |
| Stress     | Work, Academic, General      |

Table 6.3 — Recommendation database coverage by condition and cause

Each condition-cause combination provides separate recommendation sets for mild/moderate and severe severity bands, totaling 18 distinct recommendation profiles.

**Crisis Override.** When the suicidal ideation detector (Section 6.3.3) triggers, the entire recommendation lookup is bypassed. A hardcoded SUICIDAL\_REC object is returned containing four bilingual safety tips (e.g., “Remove access to any means of self-harm if possible”), the international crisis resources listed in Section 6.3.3 [29–31], and an urgent bilingual referral directing the user to contact a crisis line or visit the nearest emergency room immediately. The coping tips drawn upon throughout this engine, including box breathing and the 5-4-3-2-1 grounding exercise, follow established relaxation and grounding guidance [32, 33].

## 6.6 Cross-Platform Flutter Web Deployment

**Purpose.** To ensure maximum accessibility and support users who prefer browser-based access without installing a mobile application, the Flutter mobile client was compiled for the web and deployed to production. This web deployment provides a desktop-friendly and browser-accessible version of the entire SafeSpace application, offering supervisors and examiners a zero-installation environment to interact with the project.

**Technologies Used.** The web build compiles the Dart codebase into optimized HTML, CSS, and JavaScript. It utilizes Flutter's multi-engine rendering pipeline [26], utilizing the HTML renderer for faster initial page loads and compatibility on mobile web browsers, alongside CanvasKit (WebGL-based rendering) for desktop browsers to ensure high-performance UI rendering and smooth 60 FPS animations. The application is hosted on Netlify [28], a developer-centric cloud platform specializing in globally distributed static hosting with integrated continuous deployment.

**Deployment Architecture.**

1. **Compilation:** The app was built using the Flutter build web-- release compilation command, generating static assets in the /build/web directory (including index.html, compiled JS main entry points, manifest files, and cached assets).

2. **Hosting:** The static assets are hosted on Netlify's content delivery network (CDN) edge nodes.
3. **API Connection:** The web client communicates over HTTPS directly with the FastAPI backend hosted on Hugging Face Spaces (<https://alisakr9997-safespace.hf.space/api/v1>), maintaining a clean frontend-backend separation.
4. **Routing and Redirections:** A custom `_redirects` configuration file was added to Netlify's deployment directory to handle single-page application (SPA) routing fallbacks, routing all direct sub-page URLs back to `index.html` to allow the Flutter router to manage the internal navigation stack.

UI/UX and Functional Consistency. Since the web build compiles from the same codebase as the mobile application, the user experience is completely identical. The web version supports the same authentication flow via Supabase, daily check-ins, journal logging, goals management, and DASS-42 assessments. The responsive layout adapts the mobile navigation bar into a spacious dashboard suited for desktop and tablet screens.

---

# **CHAPTER 7**

## **CONCLUSION AND FUTURE WORK**

---

## Chapter 7: Conclusion and Future Work

### 7.1 Conclusion

This project successfully designed, implemented, and deployed SafeSpace, a cross-platform mobile application that addresses fundamental limitations in existing digital mental health tools. The system combines the clinical rigor of the DASS-42 psychometric instrument [9, 10] with the contextual sensitivity of transformer-based natural language processing [11, 14], producing a multi-modal assessment that is more resistant to self-report bias than either modality alone.

The key contributions of this project are:

**Multi-Modal Score Fusion.** The weighted fusion algorithm (60 percent survey, 40 percent text) reduces the impact of response masking by incorporating semantic analysis of the user's own words. A user who minimizes symptoms on the DASS-42 but writes a journal entry expressing significant distress will receive a fused score that reflects the true severity more accurately than the survey score alone.

**Bilingual Dialect Support.** The translation preprocessing pipeline enables the NLP engine to process colloquial Arabic dialects, including Egyptian, Levantine, and Gulf Arabic, which are the primary modes of emotional expression for millions of users in the MENA region. This capability is absent from all major commercial mental health applications reviewed in this project.

**Automated Root-Cause Extraction.** By parsing journal text against a bilingual keyword lexicon, the system identifies the primary domain of distress and delivers coping recommendations specific to that domain. A student experiencing academic stress receives different suggestions than a professional experiencing workplace burnout.

**Real-Time Crisis Safety Net.** The automated keyword scanner provides an immediate safety override when self-harm indicators are detected, connecting users with emergency resources without requiring them to explicitly request help. This proactive approach addresses a critical gap in existing mood tracking applications.

**Comprehensive Therapeutic Toolkit.** Beyond assessment, the application provides a rich set of evidence-based coping activities, including guided breathing, sensory grounding, meditation, psychoeducational content, daily wellness challenges, and cognitive distraction games, all accessible within a single, cohesive interface.

## 7.2 Future Work

Several directions for future development would extend the capabilities and clinical validity of SafeSpace:

**Biometric Data Integration.** Connecting the application to wearable device APIs such as Apple HealthKit and Google Fit would enable the system to incorporate physiological indicators, including heart rate variability, sleep duration, and physical activity levels. These objective biometric signals could serve as a third assessment modality, further reducing reliance on self-report data.

**Voice Emotion Recognition.** The existing speech-to-text functionality could be extended with audio feature analysis to detect emotional indicators in the user's vocal characteristics, such as pitch variation, speech rate, and pause patterns. This would provide an additional layer of sentiment analysis beyond the transcribed text content.

**Clinical Validation Study.** A controlled pilot study comparing the system's automated fused assessments against evaluations by licensed mental health professionals would provide empirical evidence of diagnostic accuracy. Such validation would strengthen the clinical credibility of the multi-modal fusion approach and identify areas for threshold calibration.

**Expanded Language Support.** The current translation pipeline could be extended to support additional languages and dialects, including French-Arabic code-switching common in North African countries, Urdu, and Turkish, broadening the application's accessibility across diverse populations.

**Longitudinal Trend Analysis.** Implementing time-series analysis of historical check-in and assessment data would enable the system to detect deteriorating trends over weeks or months, providing early warnings before a user reaches a critical state.

**Adaptive Fusion Weighting.** Replacing the fixed fusion weights with an adaptive weighting mechanism trained on longitudinal user data would allow the system to dynamically optimise the balance between survey and text modalities for each individual user.

**Expanded Condition Coverage.** Expanding the system to detect additional mental health conditions such as PTSD, OCD, bipolar disorder, and burnout syndrome.



---

# REFERENCES

---

### References

- [1] World Health Organization, "Depressive disorder (depression)," WHO Fact Sheets, Aug. 29, 2025. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/depression>
- [2] United Nations Children's Fund (UNICEF) Middle East and North Africa Regional Office, Integrating Mental Health and Psychosocial Support into Primary Health Care in the MENA Region: Regional Report, 2024. [Online]. Available: <https://www.unicef.org/mena/regional-country-reports-integrating-mental-health-psychosocial-support-primary-health-care>
- [3] World Health Organization and UNICEF, "WHO and UNICEF release guidance to improve access to mental health care for children and young people," Oct. 9, 2024. [Online]. Available: <https://www.who.int/news/item/09-10-2024-who-and-unicef-launch-guidance-to-improve-access-to-mental-health-care-for-children-and-young-people>
- [4] Calm.com, Inc., "Calm — The #1 App for Meditation and Sleep," 2025. [Online]. Available: <https://www.calm.com>
- [5] Headspace Inc., "Headspace: Meditation and Sleep Made Simple," 2025. [Online]. Available: <https://www.headspace.com/>
- [6] Wysa Ltd., "Wysa: Everyday Mental Health Support," 2025. [Online]. Available: <https://www.wysa.com/>
- [7] Woebot Health, "Woebot Health: Relational Agent for Mental Health," 2025. [Online]. Available: <https://woebothealth.com/>
- [8] BetterHelp, "BetterHelp: Online Counseling and Therapy," 2025. [Online]. Available: <https://www.betterhelp.com/>
- [9] S. H. Lovibond and P. F. Lovibond, *Manual for the Depression Anxiety Stress Scales*, 2nd ed. Sydney, NSW, Australia: Psychology Foundation of Australia, 1995. [Online]. Available: <https://www2.psy.unsw.edu.au/dass/>
- [10] P. F. Lovibond and S. H. Lovibond, "The structure of negative emotional states: Comparison of the Depression Anxiety Stress Scales (DASS) with the Beck Depression and Anxiety Inventories," *Behaviour Research and Therapy*, vol. 33, no. 3, pp. 335-343, Mar. 1995. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/000579679400075U>
- [11] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998-6008. [Online]. Available: <https://papers.neurips.cc/paper/7181-attention-is-all-you-need.pdf>
- [12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-*

## REFERENCES

---

HLT, 2019, pp. 4171-4186.[Online]. Available:<https://aclanthology.org/N19-1423.pdf>

[13] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv preprint arXiv:1907.11692, 2019.[Online].

Available: <https://arxiv.org/abs/1907.11692>

[14] A. Conneau et al., "Unsupervised cross-lingual representation learning at scale," in Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 8440-8451.[Online].

Available: <https://aclanthology.org/2020.acl-main.747/>

[15] A. Moujar and O. El Bourki, "MentalHealth-Darija," Hugging Face Datasets, 2025. [Online].

Available: <https://huggingface.co/datasets/moujar/MentalHealth-Darija>

[16] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proc. 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations, 2020, pp. 38-45. [Online].

Available:<https://aclanthology.org/2020.emnlp-demos.6/>

[17] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[Online]. Available: <https://arxiv.org/abs/1201.0490>

[18] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. 3rd International Conference on Learning Representations (ICLR), 2015.

[Online]. Available: <https://arxiv.org/abs/1412.6980>

[19] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in Proc. 32nd International Conference on Machine Learning (ICML), 2015, pp. 448-456.

[Online]. Available:

<https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/43442.pdf>

[20] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," Journal of Machine Learning Research, vol. 15, no. 56, pp. 1929-1958, 2014.

[Online]. Available:

<https://jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf>

[21] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2016, pp. 265-283.

[Online]. Available: <https://arxiv.org/abs/1605.08695>

## REFERENCES

---

- [22] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems* (NeurIPS), vol. 32, 2019, pp. 8024-8035.  
[Online]. Available: <https://arxiv.org/abs/1912.01703>
- [23] Google LLC, "Cloud Translation API (Google Translate)," 2025. [Online]. Available: <https://cloud.google.com/translate>
- [24] Supabase Inc., "Supabase: The Open Source Firebase Alternative," 2025. [Online]. Available: <https://supabase.com>
- [25] S. Ramírez, "FastAPI," 2025. [Online]. Available: <https://fastapi.tiangolo.com>
- [26] Google LLC, "Flutter — Build apps for any screen," 2025. [Online]. Available: <https://flutter.dev>
- [27] Hugging Face Inc., "Hugging Face Spaces," 2025. [Online]. Available: <https://huggingface.co/spaces>
- [28] Netlify Inc., "Netlify: Develop, Preview, Ship," 2025. [Online]. Available: <https://www.netlify.com>
- [29] International Association for Suicide Prevention (IASP), "Crisis Centres," 2025. [Online]. Available: [https://www.iasp.info/resources/Crisis\\_Centres](https://www.iasp.info/resources/Crisis_Centres)
- [30] Crisis Text Line, "Crisis Text Line: Text HOME to 741741," 2025. [Online]. Available: <https://www.crisistextline.org>
- [31] Befrienders Worldwide, "Befrienders Worldwide: Emotional Support Helplines," 2025. [Online]. Available: <https://www.befrienders.org>
- [32] University of Rochester Medical Center, Behavioral Health Partners, "5-4-3-2-1 Coping Technique for Anxiety," 2018. [Online]. Available: <https://www.urmc.rochester.edu/behavioral-health-partners/bhp-blog/april-2018/5-4-3-2-1-coping-technique-for-anxiety>
- [33] National Center for Complementary and Integrative Health (NCCIH), "Relaxation Techniques for Health," U.S. National Institutes of Health, 2024. [Online]. Available: <https://www.nccih.nih.gov/health/relaxation-techniques-for-health>